

Aurélien Géron — *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*
Azərbaycan dilinə tərcümə · texniki terminlər ingilis dilində saxlanılıb

Fəsil 14. Convolutional Neural Network-lərlə Dərin Computer Vision

IBM-in Deep Blue superkompüterləri hələ 1996-cı ildə şahmat üzrə dünya çempionu Garry Kasparovu məğlub etsə də, kompüterlərin şəkildə bir kiçik itir aşkarlanması və ya danışılan sözlərin tanınması kimi görünüşcə adi tapşırıqları etibarlı şəkildə yerinə yetirə bilməsi yalnız nisbətən bu yaxınlarda mümkün oldu. Bəs bu tapşırıqlar biz insanlar üçün niyə bu qədər asandır? Cavab ondadır ki, qavrayış əsasən şüurumuzun həddlərindən kənarda — beynimizdəki ixtisaslaşmış vizual, eşitmə və digər sensor modularda baş verir. Sensor məlumat şüurumuza çatana qədər o, artıq yüksək səviyyəli xüsusiyyətlərlə (high-level features) bəzədilmiş olur; məsələn, sevimli bir itir şəklinə baxanda onu görməməyi, onun sevimliliyini sezməməyi seçə bilmirsiniz. Sevimli bir itir necə tanıdığınızı da izah edə bilmirsiniz; bu sizə sadəcə bəllidir. Beləliklə, subyektiv təcrübəmizə etibar edə bilmərik: qavrayış əsla adi deyil və onu anlamaq üçün sensor modullarımızın necə işlədiyinə baxmalıyıq.

Convolutional neural network-lər (CNN-lər) beynin vizual korteksinin (visual cortex) öyrənilməsindən yarandı və 1980-ci illərdən bəri kompüter şəkil tanınmasında (image recognition) istifadə olunur. Son 10 ildə hesablama gücünün artması, mövcud təlim (training) məlumatlarının həcmi və dərin şəbəkələrin (deep nets) təlimi üçün Fəsil 11-də təqdim olunan üsullar sayəsində CNN-lər bəzi mürəkkəb vizual tapşırıqlarda insandan üstün (superhuman) performansla nail olmuşlar. Onlar şəkil axtarış xidmətlərini, özünü idarə edən avtomobilləri (self-driving cars), avtomatik video təsnifat sistemlərini və daha çoxunu işlədir. Üstəlik, CNN-lər vizual qavrayışla məhdudlaşmır: onlar səs tanınması (voice recognition) və təbii dil emalı (natural language processing) kimi bir çox digər tapşırıqda da uğurludur. Lakin hələlik vizual tətbiqlərə diqqət yetirəcəyik.

Bu fəsildə CNN-lərin haradan gəldiyini, onların qurucu bloklarının (building blocks) necə göründüyünü və onları Keras-dan istifadə edərək necə tətbiq edəcəyimizi araşdıracağıq. Sonra ən yaxşı CNN arxitekturalarından bəzilərini, eləcə də digər vizual tapşırıqları — object detection (şəkildə birdən çox obyektin təsnif edilməsi və onların ətrafına bounding box yerləşdirilməsi) və semantic segmentation (hər pikselin aid olduğu obyektin sinfinə görə təsnif edilməsi) daxil olmaqla — müzakirə edəcəyik.

Vizual Korteksin Arxitekturası

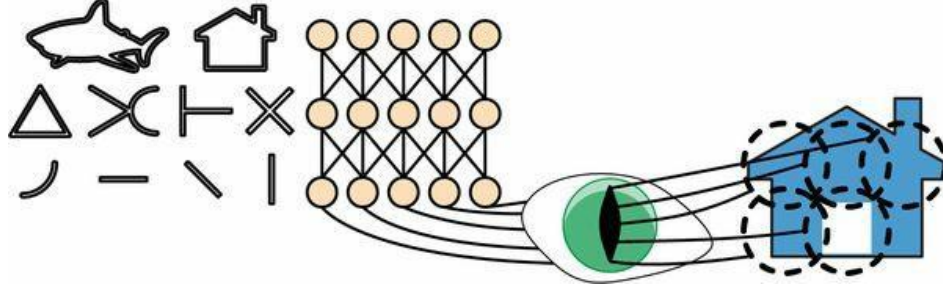
David H. Hubel və Torsten Wiesel 1958 və 1959-cu illərdə pişiklər¹ (və bir neçə il sonra meymunlar²) üzərində bir sıra təcrübələr³ apararaq vizual korteksin quruluşu haqqında mühüm

¹David H. Hubel, "Single Unit Activity in Striate Cortex of Unrestrained Cats", The Journal of Physiology 147 (1959): 226–238.

²David H. Hubel and Torsten N. Wiesel, "Receptive Fields and Functional Architecture of Monkey Striate Cortex", The Journal of Physiology 195 (1968): 215–243.

³David H. Hubel and Torsten N. Wiesel, "Receptive Fields of Single Neurons in the Cat's Striate Cortex", The Journal of Physiology 148 (1959): 574–591.

fikirilər verdilər (müəlliflər bu işə görə 1981-ci ildə Fiziologiya və ya Tibb üzrə Nobel mükafatına layiq görüldülər). Xüsusilə, onlar göstərdilər ki, vizual korteksdəki bir çox neyronun kiçik *local receptive field*-i var, yəni onlar yalnız vizual sahənin məhdud bir bölgəsində yerləşən vizual stimullara reaksiya verir (bax Şəkil 14-1; orada beş neyronun local receptive field-i kəşik dairələrlə təsvir olunub). Müxtəlif neyronların receptive field-ləri bir-birinin üstünə düşə bilər və onlar birlikdə bütün vizual sahəni döşəyir.



Şəkil 14-1. Vizual korteksdəki bioloji neyronlar vizual sahənin receptive field adlanan kiçik bölgələrindəki müəyyən nümunələrə (patterns) reaksiya verir; vizual signal ardıcıl beyin modullarından keçdikcə neyronlar daha böyük receptive field-lərdə daha mürəkkəb nümunələrə reaksiya verir

Bundan əlavə, müəlliflər göstərdilər ki, bəzi neyronlar yalnız üfüqi xətt şəkillərinə reaksiya verir, digərləri isə yalnız fərqli istiqamətli xətlərə reaksiya verir (iki neyronun eyni receptive field-i ola, lakin fərqli xətt istiqamətlərinə reaksiya verə bilər). Onlar həmçinin qeyd etdilər ki, bəzi neyronların receptive field-ləri daha böyükdür və onlar aşağı səviyyəli nümunələrin kombinasiyaları olan daha mürəkkəb nümunələrə reaksiya verir. Bu müşahidələr belə bir fikrə gətirib çıxardı ki, yüksək səviyyəli neyronlar qonşu aşağı səviyyəli neyronların çıxışlarına əsaslanır (Şəkil 14-1-də qeyd edin ki, hər neyron yalnız əvvəlki qatdakı yaxın neyronlara bağlıdır). Bu güclü arxitektura vizual sahənin istənilən hissəsində hər cür mürəkkəb nümunəni aşkarlamağa qadirdir.

Vizual korteksin bu tədqiqatları 1980-ci ildə təqdim olunan və tədricən indi convolutional neural network adlandırdığımız şeyə çevrilən *neocognitron*-u⁴ ruhlandırdı. Mühüm bir mərhələ Yann LeCun və başqalarının 1998-ci il məqaləsi⁵ oldu; orada banklar tərəfindən çeklərdəki əlyazma rəqəmlərini tanımaq üçün geniş istifadə olunan məşhur LeNet-5 arxitekturası təqdim olundu. Bu arxitektura artıq tanıdığınız bəzi qurucu bloklar var — məsələn, fully connected layer-lər və sigmoid activation function-lar, lakin o, həm də iki yeni qurucu blok təqdim edir: convolutional layer-lər və pooling layer-lər. Gəlin indi onlara baxaq.

QEYD

Bəs niyə şəkil tanınması tapşırıqları üçün sadəcə fully connected layer-li dərin neural network istifadə etməyəək? Təəssüf ki, bu, kiçik şəkillər (məsələn, MNIST) üçün yaxşı işləsə də, böyük şəkillər üçün tələb etdiyi nəhəng sayda parametmə görə dağılır. Məsələn, 100×100 piksellə şəkilə 10.000 piksel var və əgər ilk qatda cəmi 1.000 neyron olsa (ki, bu da artıq növbəti qata ötürülən məlumatın həcmi ciddi şəkildə məhdudlaşdırır), bu, ümumilikdə 10

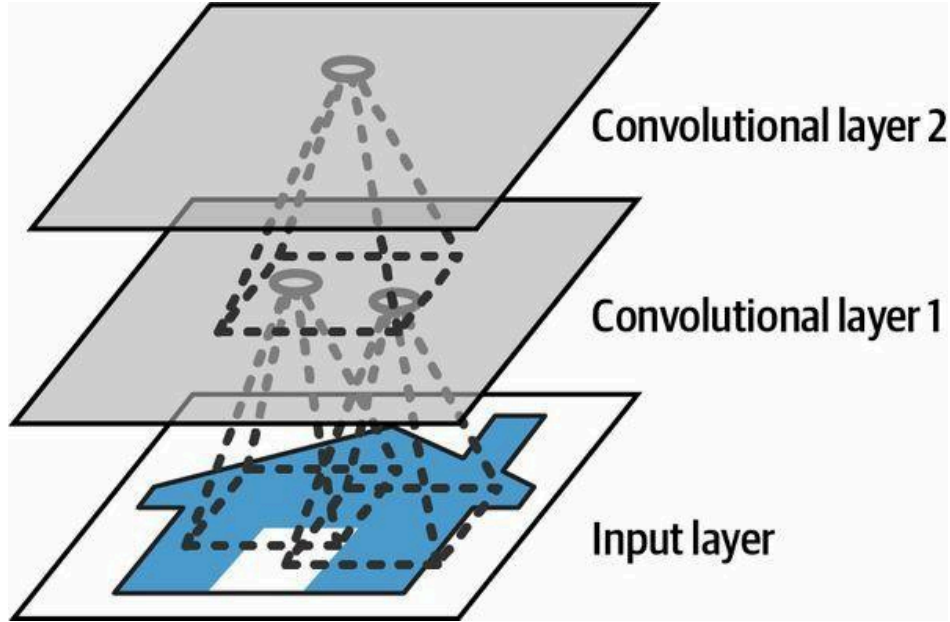
⁴Kunihiko Fukushima, "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position", *Biological Cybernetics* 36 (1980): 193–202.

⁵Yann LeCun et al., "Gradient-Based Learning Applied to Document Recognition", *Proceedings of the IEEE* 86, no. 11 (1998): 2278–2324.

milyon bağlantı deməkdir. Və bu, sadəcə birinci qatdır. CNN-lər bu problemi qismən bağlı (partially connected) layer-lərdən və weight sharing-dən istifadə edərək həll edir.

Convolutional Layer-lər

CNN-in ən vacib qurucu bloku convolutional layer-dir:⁶ birinci convolutional layer-dəki neyronlar giriş şəklindəki hər bir piksellə bağlı deyil (əvvəlki fəsillərdə müzakirə olunan qatlarda olduğu kimi), yalnız öz receptive field-lərindəki piksellərlə bağlıdır (bax Şəkil 14-2). Öz növbəsində, ikinci convolutional layer-dəki hər neyron yalnız birinci qatdakı kiçik düzbucaqlı içində yerləşən neyronlara bağlıdır. Bu arxitektura şəbəkəyə birinci gizli qatda kiçik aşağı səviyyəli xüsusiyyətlərə (low-level features) diqqət yetirməyə, sonra onları növbəti gizli qatda daha böyük yüksək səviyyəli xüsusiyyətlərə birləşdirməyə və s. imkan verir. Bu iyerarxik quruluş real dünya şəkillərində geniş yayılıb ki, bu da CNN-lərin şəkil tanınması üçün niyə bu qədər yaxşı işlədiyinin səbəblərindən biridir.



Şəkil 14-2. Düzbucaqlı local receptive field-li CNN layer-ləri

QEYD

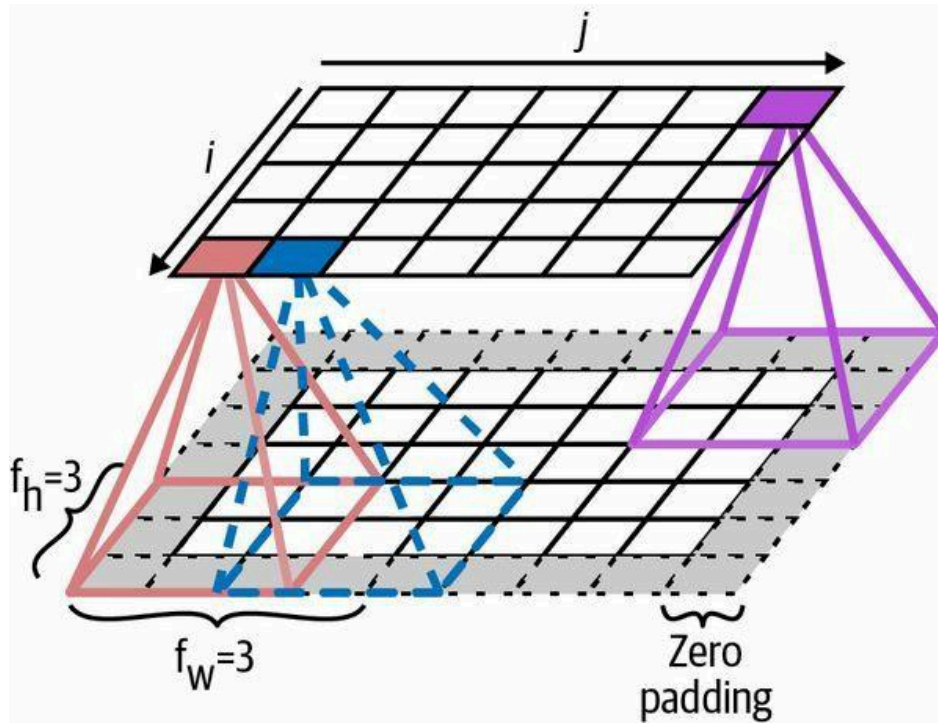
İndiyə qədər baxdığımız bütün çoxqatlı neural network-lərdə qatlar neyronların uzun bir sırasından ibarət idi və şəkilləri neural network-ə vermədən əvvəl onları 1D-yə düzləşdirməli (flatten) idik. CNN-də isə hər qat 2D şəklində təmsil olunur ki, bu da neyronları onların müvafiq girişləri ilə uyğunlaşdırmağı asanlaşdırır.

Müəyyən bir qatın i sətirində, j sütununda yerləşən neyron əvvəlki qatda i -dən $i + f_h - 1$ -ə qədər sətirlərdə, j -dən $j + f_w - 1$ -ə qədər sütunlarda yerləşən neyronların çıxışlarına bağlıdır; burada f_h və f_w receptive field-in hündürlüyü və enidir (bax Şəkil 14-3). Bir qatın əvvəlki qatla eyni

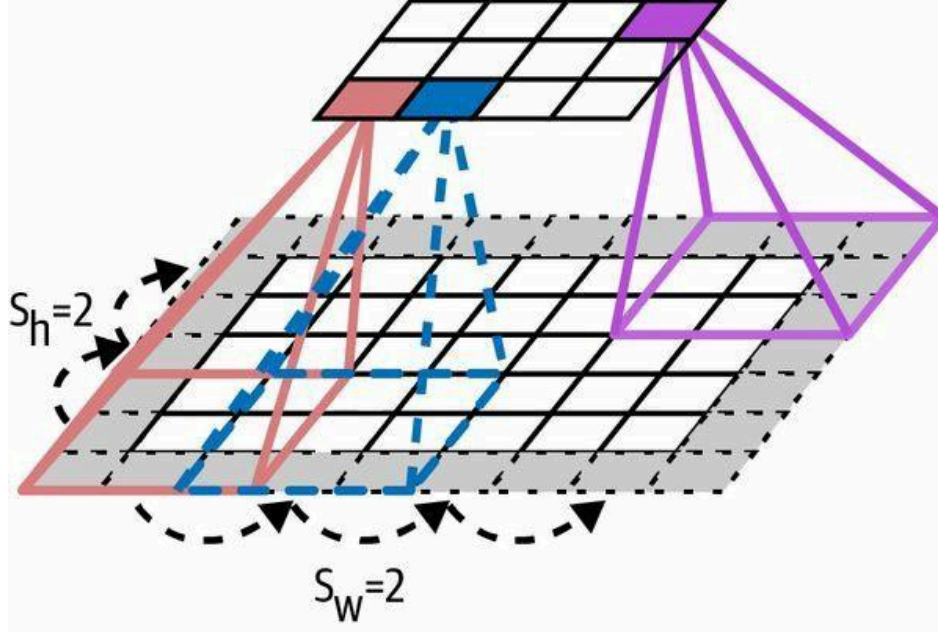
⁶Convolution iki funksiyayı bir-birinin üzərində sürüşdürən və onların nöqtəvi hasilinin (pointwise multiplication) integralını ölçən riyazi əməliyyatdır. Onun Fourier transform və Laplace transform ilə dərin əlaqələri var və signal emalında geniş istifadə olunur. Convolutional layer-lər əslində convolution-a çox bənzəyən cross-correlation-dan istifadə edir (ətraflı məlumat üçün bax: <https://homl.info/76>).

hündürlük və enə malik olması üçün adətən girişlərin ətrafına sıfırlar əlavə olunur (diaqramda göstərildiyi kimi). Buna *zero padding* deyilir.

Böyük bir giriş qatını receptive field-ləri aralı yerləşdirməklə daha kiçik bir qata bağlamaq da mümkündür (Şəkil 14-4-də göstərildiyi kimi). Bu, modelin hesablama mürəkkəbliyini kəskin şəkildə azaldır. Bir receptive field-dən digərinə üfüqi və ya şaquli addım ölçüsünə *stride* deyilir. Diaqramda 5×7 giriş qatı (üstəgəl zero padding) 3×3 receptive field-lərdən və 2 stride-dan istifadə etməklə 3×4 qata bağlanıb (bu nümunədə stride hər iki istiqamətdə eynidir, lakin belə olmaq məcburiyyətində deyil). Yuxarı qatda i sətirində, j sütununda yerləşən neyron əvvəlki qatda $i \times s_h$ -dən $i \times s_h + f_h - 1$ -ə qədər sətirlərdə, $j \times s_w$ -dən $j \times s_w + f_w - 1$ -ə qədər sütunlarda yerləşən neyronların çıxışlarına bağlıdır; burada s_h və s_w şaquli və üfüqi stride-lərdir.



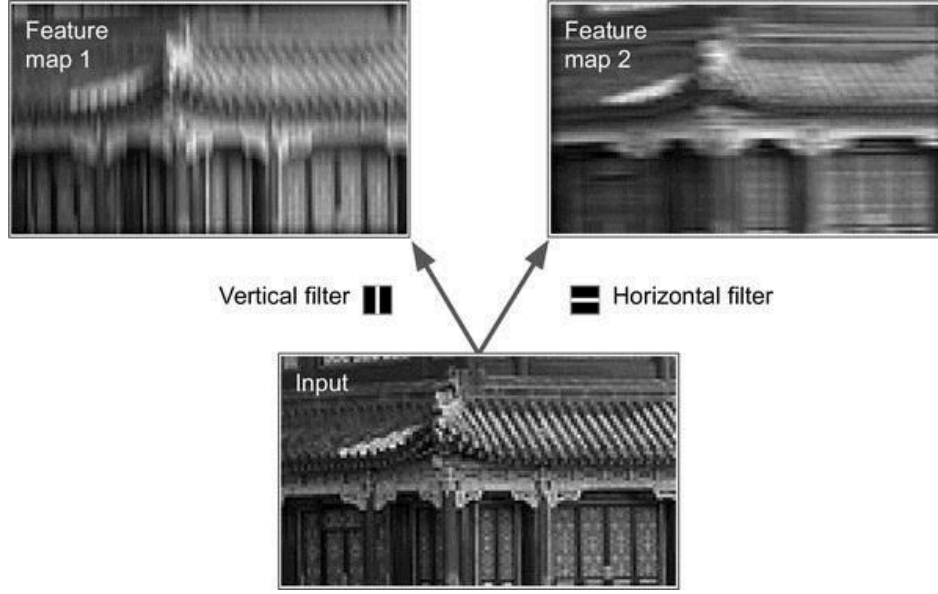
Şəkil 14-3. Layer-lər arasındakı bağlantılar və zero padding



Şəkil 14-4. Stride 2 ilə ölçünün (dimensionality) azaldılması

Filter-lər

Bir neyronun çəkirlərini (weights) receptive field ölçüsündə kiçik bir şəkil kimi təsvir etmək olar. Məsələn, Şəkil 14-5 iki mümkün çəki dəstini göstərir; bunlara *filter* (və ya *convolution kernel*, ya da sadəcə *kernel*) deyilir. Birincisi ortasında şaquli ağ xətti olan qara kvadrat kimi təsvir olunub (bu, mərkəzi sütunu istisna olmaqla tamamilə 0-larla dolu 7×7 matrisdir, mərkəzi sütun isə tamamilə 1-lərlə doludur); bu çəkildən istifadə edən neyronlar mərkəzi şaquli xəttədən başqa öz receptive field-lərindəki hər şeyə məhəl qoymayacaq (çünki mərkəzi şaquli xəttədekilər istisna olmaqla bütün girişlər 0-a vurulacaq). İkinci filter ortasında üfüqi ağ xətti olan qara kvadratdır. Bu çəkildən istifadə edən neyronlar mərkəzi üfüqi xəttədən başqa öz receptive field-lərindəki hər şeyə məhəl qoymayacaq.

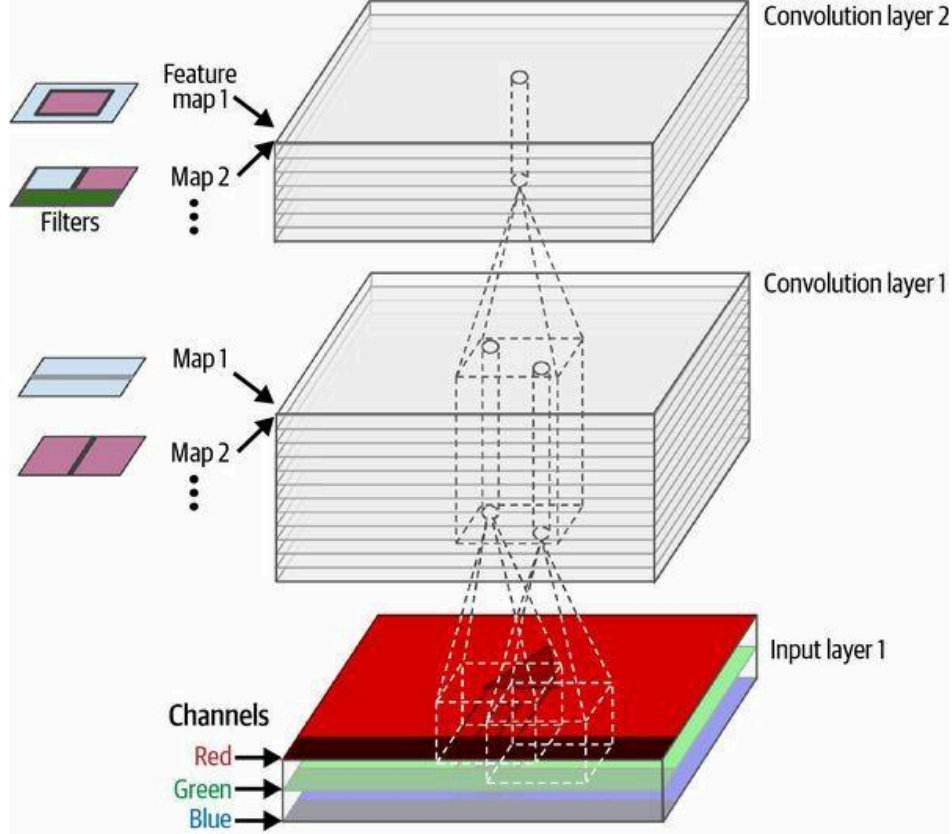


Şəkil 14-5. İki feature map almaq üçün iki fərqli filter-in tətbiqi

İndi əgər bir qatdakı bütün neyronlar eyni şaquli xətt filter-indən (və eyni bias term-dən) istifadə edərsə və şəbəkəyə Şəkil 14-5-də göstərilən giriş şəklini (aşağıdakı şəkli) versəniz, qat yuxarı sol şəkli çıxaracaq. Diqqət edin ki, şaquli ağ xətlər gücləndirilir, qalan hissə isə bulanıqlaşır. Eyni şəkildə, bütün neyronlar eyni üfüqi xətt filter-indən istifadə edərsə, yuxarı sağ şəkli alarsınız; diqqət edin ki, üfüqi ağ xətlər gücləndirilir, qalanı isə bulanıqlaşır. Beləliklə, eyni filter-dən istifadə edən neyronlarla dolu bir qat şəkildə filter-i ən çox aktivləşdirən sahələri vurğulayan bir *feature map* çıxarır. Lakin narahat olmayın, filter-ləri əl ilə təyin etməyə ehtiyac olmayacaq: əvəzində, təlim zamanı convolutional layer öz tapşırığı üçün ən faydalı filter-ləri avtomatik öyrənəcək, yuxarıdakı qatlar isə onları daha mürəkkəb nümunələrə birləşdirməyi öyrənəcək.

Çoxsaylı Feature Map-ləri Yığmaq

İndiyə qədər sadəlik üçün hər convolutional layer-in çıxışını 2D qat kimi təsvir etdim, amma əslində convolutional layer-in çoxsaylı filter-ləri var (sayını siz qərarlaşdırırsınız) və o, filter başına bir feature map çıxarır, ona görə də daha dəqiq olaraq 3D şəkildə təsvir olunur (bax Şəkil 14-6). Hər feature map-də hər piksel üçün bir neyron var və müəyyən bir feature map daxilindəki bütün neyronlar eyni parametrləri (yəni eyni kernel və bias term-i) paylaşır. Müxtəlif feature map-lərdəki neyronlar fərqli parametrlərdən istifadə edir. Bir neyronun receptive field-i əvvəl təsvir olunduğu kimidir, lakin o, əvvəlki qatın bütün feature map-lərinə yayılır. Qısacası, convolutional layer eyni anda öz girişlərinə çoxsaylı öyrənilə bilən filter tətbiq edir ki, bu da onu girişlərin istənilən yerində çoxsaylı xüsusiyyəti aşkarlamağa qadir edir.



Şəkil 14-6. Üç rəng kanalı (color channel) olan rəngli şəkli emal edən, hər biri çoxlu filter-ə (kernel) malik iki convolutional layer; hər convolutional layer filter başına bir feature map çıxarır

QEYD

Bir feature map-dəki bütün neyronların eyni parametrləri paylaşması faktı modeldəki parametrlərin sayını kəskin şəkildə azaldır. CNN bir yerdə nümunəni tanımağı öyrəndikdən sonra onu istənilən başqa yerdə də tanıya bilər. Bunun əksinə, fully connected neural network bir yerdə nümunəni tanımağı öyrəndikdən sonra onu yalnız həmin konkret yerdə tanıya bilər.

Giriş şəkilləri də çoxsaylı alt qatlardan ibarətdir: hər rəng kanalı (color channel) üçün bir. Fəsil 9-da qeyd olunduğu kimi, adətən üç kanal olur: qırmızı, yaşıl və mavi (RGB). Boz-ağ (grayscale) şəkillərin yalnız bir kanalı var, lakin bəzi şəkillərin daha çoxu ola bilər — məsələn, əlavə işıq tezliklərini (infraqırmızı kimi) tutan peyk şəkilləri.

Daha dəqiq desək, müəyyən bir convolutional layer l-də k feature map-inin i sətirində, j sütununda yerləşən neyron əvvəlki $l - 1$ qatındakı neyronların $i \times s_h$ -dən $i \times s_h + f_h - 1$ -ə qədər sətirlərdə və $j \times s_w$ -dən $j \times s_w + f_w - 1$ -ə qədər sütunlarda yerləşən, bütün feature map-lər boyunca ($l - 1$ qatında) — çıxışlarına bağlıdır. Diqqət edin ki, bir qat daxilində eyni i sətirində və j sütununda, lakin fərqli feature map-lərdə yerləşən bütün neyronlar əvvəlki qatda tam olaraq eyni neyronların çıxışlarına bağlıdır.

Tənlik 14-1 yuxarıdakı izahları bir böyük riyazi tənlikdə yekunlaşdırır: o, convolutional layer-dəki müəyyən bir neyronun çıxışını necə hesablamağı göstərir. Bütün fərqli indekslərə görə bir az

çirkin görünür, lakin etdiyi tək şey bütün girişlərin çəkili cəmini, üstəgəl bias term-i hesablamaqdır.

Tənlik 14-1. Convolutional layer-dəki bir neyronun çıxışının hesablanması

$$z_{i,j,k} = b_k + \sum_{(u=0 \dots f_n-1)} \sum_{(v=0 \dots f_w-1)} \sum_{(k'=0 \dots f_n-1)}$$

$$x_{i',j',k'} \times w_{u,v,k',k}$$

$$\text{burada } i' = i \times s_n + u, \quad j' = j \times s_w + v$$

Bu tənlikdə:

- $z_{\{i,j,k\}}$ convolutional layer-də (l qatı) k feature map-inin i sətirində, j sütununda yerləşən neyronun çıxışıdır.
- Əvvəl izah olunduğu kimi, s_n və s_w şaquli və üfüqi stride-lərdir, f_n və f_w receptive field-in hündürlüyü və enidir, f_n isə əvvəlki qatda ($l - 1$ qatı) feature map-lərin sayıdır.
- $x_{\{i',j',k'\}}$ $l - 1$ qatında, i' sətirində, j' sütununda, k' feature map-ində (və ya əvvəlki qat giriş qatıdırsa, k' kanalında) yerləşən neyronun çıxışıdır.
- b_k k feature map-i üçün (l qatında) bias term-dir. Onu k feature map-inin ümumi parlaqlığını tənzimləyən bir düymə kimi düşünə bilərsiniz.
- $w_{\{u,v,k',k\}}$ l qatının k feature map-indəki istənilən neyron ilə onun receptive field-inə nisbətən u sətirində, v sütununda yerləşən girişi (və k' feature map-i) arasındakı bağlantı çəkisidir (connection weight).

Gəlin indi Keras-dan istifadə edərək convolutional layer-i necə yaratmağı və işlətməyi görək.

Convolutional Layer-ləri Keras ilə Tətbiq Etmək

Əvvəlcə Scikit-Learn-in `load_sample_image()` funksiyasından və Keras-ın `CenterCrop` və `Rescaling` layer-lərindən (hamısı Fəsil 13-də təqdim olunub) istifadə edərək bir neçə nümunə şəkli yükləyib emal edək:

```
from sklearn.datasets import load_sample_images
import tensorflow as tf

images = load_sample_images()["images"]
images = tf.keras.layers.CenterCrop(height=70, width=120)(images)
images = tf.keras.layers.Rescaling(scale=1 / 255)(images)
```

`images` tensorunun formasına (`shape`) baxaq:

```
>>> images.shape
TensorShape([2, 70, 120, 3])
```

Hmm, bu 4D tensordur; bunu əvvəl görməmişdik! Bütün bu ölçülər nə deməkdir? Birincisi, iki nümunə şəkil var ki, bu da birinci ölçünü izah edir. Sonra hər şəkil 70×120 -dir, çünki `CenterCrop` layer-ini yaradanda təyin etdiyimiz ölçü budur (orijinal şəkillər 427×640 idi). Bu, ikinci və üçüncü ölçüləri izah edir. Nəhayət, hər piksel rəng kanalı başına bir dəyər saxlayır və onlardan üçü var — qırmızı, yaşıl və mavi — ki, bu da son ölçünü izah edir.

İndi 2D convolutional layer yaradaq və nə alındığını görmək üçün ona bu şəkilləri verək. Bunun üçün Keras `Convolution2D` layer-ini (alias `Conv2D`) təqdim edir. Daxildə bu qat TensorFlow-un `tf.nn.conv2d()` əməliyyatına əsaslanır. Gəlin hər biri 7×7 ölçüsündə 32 filter-li convolutional

layer yaradaq (`kernel_size=7` istifadə edirik ki, bu da `kernel_size=(7, 7)` ilə eynidir) və bu qatı iki şəkildən ibarət kiçik batch-ə tətbiq edək:

```
conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7)
fmaps = conv_layer(images)
```

QEYD

2D convolutional layer-dən danışdığımız zaman “2D” məkan ölçülərinin (height və width) sayına aiddir, lakin gördüyünüz kimi qat 4D girişlər qəbul edir: gördüyümüz kimi, iki əlavə ölçü batch ölçüsü (birinci ölçü) və kanallardır (channels, son ölçü).

İndi çıxışın formasına baxaq:

```
>>> fmaps.shape
TensorShape([2, 64, 114, 32])
```

Çıxış forması giriş forması ilə oxşardır, iki əsas fərqlə. Birincisi, 3 əvəzinə 32 kanal var. Bunun səbəbi `filters=32` təyin etməyimizdir, ona görə də 32 çıxış feature map alırıq: hər yerdə qırmızı, yaşıl və mavinin intensivliyi əvəzinə indi hər yerdə hər xüsusiyyətin intensivliyini alırıq. İkincisi, hündürlük və en hər ikisi 6 piksel kiçilib. Bu, `Conv2D` layer-inin susmaya görə (default) heç bir zero-padding istifadə etməməsi ilə bağlıdır ki, bu da filter ölçüsündən asılı olaraq çıxış feature map-lərinin kənarlarında bir neçə piksel itirməyimizə səbəb olur. Bu halda, kernel ölçüsü 7 olduğu üçün üfüqi 6 piksel və şaquli 6 piksel (yəni hər tərəfdən 3 piksel) itiririk.

XƏBƏRDARLIQ

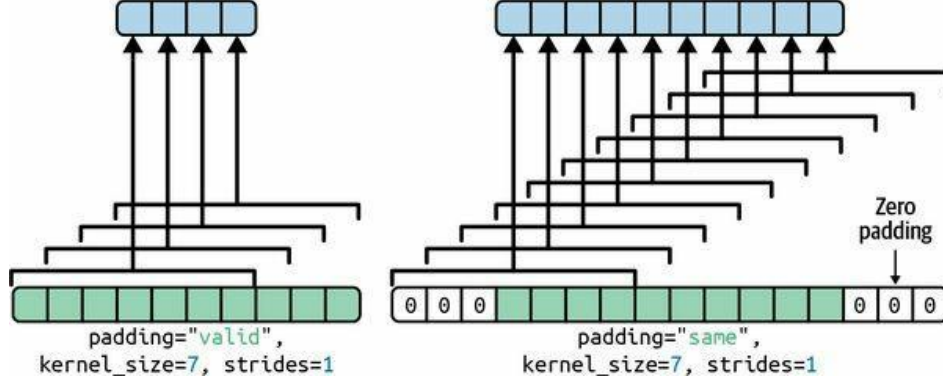
Susmaya görə seçim təəccüblü şəkildə `padding="valid"` adlanır, ki bu da əslində heç bir zero-padding olmaması deməkdir! Bu ad ondan irəli gəlir ki, bu halda hər neyronun receptive field-i girişin daxilindəki etibarlı (valid) mövqelərdə tam olaraq yerləşir (sərhəddən kənara çıxmır). Bu, Keras-a xas bir qəribəlik deyil: hamı bu qəribə terminologiyadan istifadə edir.

Əgər əvəzində `padding="same"` təyin etsək, onda girişlər çıxış feature map-lərinin girişlərlə eyni ölçüdə olmasını təmin etmək üçün hər tərəfdən kifayət qədər sıfırla doldurulur (bu seçimin adı da buradan gəlir):

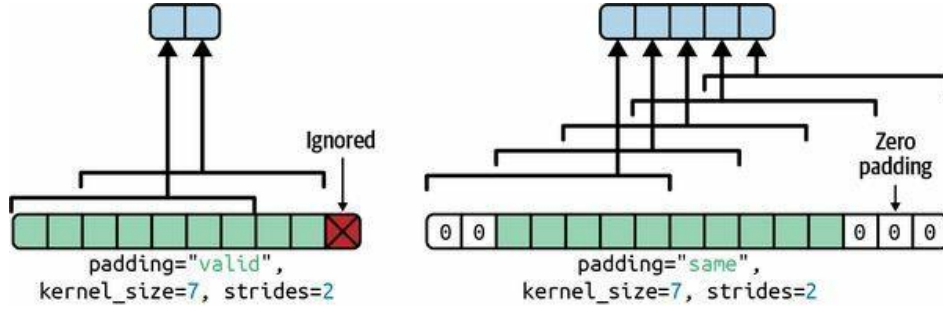
```
>>> conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7,
...                                     padding="same")
...
>>> fmaps = conv_layer(images)
>>> fmaps.shape
TensorShape([2, 70, 120, 32])
```

Bu iki padding seçimi Şəkil 14-7-də göstərilib. Sadəlik üçün burada yalnız üfüqi ölçü göstərilir, lakin əlbəttə eyni məntiq şaquli ölçüyə də aiddir.

Əgər `stride` 1-dən böyükdürsə (istənilən istiqamətdə), onda `padding="same"` olsa belə çıxış ölçüsü giriş ölçüsünə bərabər olmayacaq. Məsələn, `strides=2` (və ya ekvivalent olaraq `strides=(2, 2)`) təyin etsəniz, çıxış feature map-ləri 35×60 olacaq: şaquli və üfüqi hər ikisi yarıya bölünüb. Şəkil 14-8 hər iki padding seçimi ilə `strides=2` olduqda nə baş verdiyini göstərir.



Şəkil 14-7. strides=1 olduqda iki padding seçimi



Şəkil 14-8. Stride 1-dən böyük olduqda, "same" padding istifadə edilsə belə, çıxış xeyli kiçik olur ("valid" padding isə bəzi girişləri nəzərə almaya bilər)

Maraqlanırsınızsa, çıxış ölçüsü belə hesablanır:

- padding="valid" ilə, əgər girişin eni i_h -dirsə, onda çıxış eni $(i_h - f_h + s_h) / s_h$ -a bərabərdir (aşağıya yuvarlaqlaşdırılır). Xatırlayın ki, f_h kernel eni, s_h isə üfüqi stride-dır. Bölmədəki istənilən qalıq giriş şəklinin sağ tərəfindəki nəzərə alınmayan sütunlara uyğun gəlir. Eyni məntiqə çıxış hündürlüyünü və şəklın altındakı istənilən nəzərə alınmayan sətirləri hesablamaq olar.
- padding="same" ilə, çıxış eni i_h / s_h -a bərabərdir (yuxarıya yuvarlaqlaşdırılır). Bunu mümkün etmək üçün giriş şəklının sol və sağına lazımi sayda sıfır sütun əlavə olunur (mümkünsə bərabər sayda, ya da sağ tərəfə sadəcə bir dənə çox). Çıxış eni o_h -dirsə, əlavə olunan sıfır sütunların sayı $(o_h - 1) \times s_h + f_h - i_h$ -dir. Yenə də eyni məntiqə çıxış hündürlüyünü və əlavə olunan sıfır sətirlərin sayını hesablamaq olar.

İndi qatın çəkilərinə baxaq (Tənlik 14-1-də w və b kimi qeyd olunmuşdu). Dense layer kimi, Conv2D layer-i də qatın bütün çəkilərini, o cümlədən kernel-ləri və bias-ları saxlayır. Kernel-lər təsadüfi şəkildə (random) ilkinləşdirilir (initialize), bias-lar isə sıfıra ilkinləşdirilir. Bu çəkilərə weights atributu vasitəsilə TF dəyişənləri kimi, ya da get_weights() metodu vasitəsilə NumPy massivləri kimi daxil olmaq mümkündür:

```
>>> kernels, biases = conv_layer.get_weights()
>>> kernels.shape
(7, 7, 3, 32)
>>> biases.shape
(32,)
```

kernels massivi 4D-dir və onun forması $[\text{kernel_height}, \text{kernel_width}, \text{input_channels}, \text{output_channels}]$ -dir. biases massivi 1D-dir, forması $[\text{output_channels}]$ -dir. Çıxış kanallarının sayı çıxış feature map-lərinin sayına, o da filter-lərin sayına bərabərdir.

Ən vacibi, qeyd edin ki, giriş şəkillərinin hündürlüyü və eni kernel-in formasında görünür: bunun səbəbi əvvəl izah olunduğu kimi çıxış feature map-lərindəki bütün neyronların eyni çəkirləri paylaşmasıdır. Bu o deməkdir ki, bu qata istənilən ölçüdə şəkil verə bilərsiniz, bir şərtlə ki, onlar ən azı kernel-lər qədər böyük olsun və düzgün sayda kanala malik olsun (bu halda üç).

Nəhayət, Conv2D layer yaradanda adətən activation function (məsələn, ReLU) təyin etmək, həmçinin müvafiq kernel initializer-i (məsələn, He initialization) təyin etmək istəyəcəksiniz. Bu, Dense layer-lər üçün olduğu kimi eyni səbəbdəndir: convolutional layer xətti əməliyyat yerinə yetirir, ona görə də əgər çoxsaylı convolutional layer-i activation function olmadan yığsanız, onların hamısı bir tək convolutional layer-ə ekvivalent olardı və onlar həqiqətən mürəkkəb bir şey öyrənə bilməzdilər.

Gördüyünüz kimi, convolutional layer-lərin kifayət qədər çox hiperparametri var: filters, kernel_size, padding, strides, activation, kernel_initializer və s. Həmişə olduğu kimi, düzgün hiperparametr dəyərlərini tapmaq üçün cross-validation istifadə edə bilərsiniz, lakin bu, çox vaxt aparır. Hansı hiperparametr dəyərlərinin praktikada ən yaxşı işlədiyi barədə təsəvvür yaratmaq üçün bu fəsildə bir az sonra ümumi CNN arxitekturalarını müzakirə edəcəyik.

Yaddaş Tələbləri

CNN-lərlə bağlı başqa bir çətinlik odur ki, convolutional layer-lər nəhəng həcmdə RAM tələb edir. Bu, xüsusilə təlim zamanı doğrudur, çünki backpropagation-un geri keçidi (reverse pass) forward pass zamanı hesablanan bütün aralıq dəyərləri tələb edir.

Məsələn, stride 1 və "same" padding ilə 200 ədəd 5×5 filter-li convolutional layer-i nəzərdən keçirək. Əgər giriş 150×100 RGB şəkildirsə (üç kanal), onda parametrlərin sayı $(5 \times 5 \times 3 + 1) \times 200 = 15.200$ -dür (+ 1 bias term-lərinə uyğundur) ki, bu da fully connected layer ilə müqayisədə kifayət qədər kiçikdir.⁷ Lakin 200 feature map-in hər biri 150×100 neyron saxlayır və bu neyronların hər biri öz $5 \times 5 \times 3 = 75$ girişinin çəkili cəmini hesablamalıdır: bu, ümumilikdə 225 milyon float vurma deməkdir. Fully connected layer qədər pis deyil, lakin yenə də kifayət qədər hesablama tutumludur. Üstəlik, əgər feature map-lər 32-bitlik float ilə təmsil olunarsa, onda convolutional layer-in çıxışı $200 \times 150 \times 100 \times 32 = 96$ milyon bit (12 MB)⁸ RAM tutacaq. Və bu, sadəcə bir nümunə üçündür — əgər təlim batch-ində 100 nümunə varsa, onda bu qat 1,2 GB RAM istifadə edəcək!

Çıxarış zamanı (inference, yəni yeni nümunə üçün proqnoz verərkən) bir qatın tutduğu RAM növbəti qat hesablanan kimi azad oluna bilər, ona görə də yalnız ardıcıl iki qatın tələb etdiyi qədər RAM-a ehtiyacınız var. Lakin təlim zamanı forward pass-da hesablanan hər şey reverse pass üçün saxlanmalıdır, ona görə də lazım olan RAM həcmi (ən azı) bütün qatların tələb etdiyi ümumi RAM həcminə bərabərdir.

⁷Eyni ölçülü çıxışlar yaratmaq üçün fully connected layer $200 \times 150 \times 100$ neyrona ehtiyac duyardı, hər biri bütün $150 \times 100 \times 3$ girişə bağlı olardı. Onun $200 \times 150 \times 100 \times (150 \times 100 \times 3 + 1) \approx 135$ milyard parametri olardı!

⁸Beynəlxalq vahidlər sisteminə (SI) $1 \text{ MB} = 1.000 \text{ KB} = 1.000 \times 1.000 \text{ bayt} = 1.000 \times 1.000 \times 8 \text{ bit}$. Və $1 \text{ MiB} = 1.024 \text{ KiB} = 1.024 \times 1.024 \text{ bayt}$. Beləliklə, $12 \text{ MB} \approx 11,44 \text{ MiB}$.

İPUCU

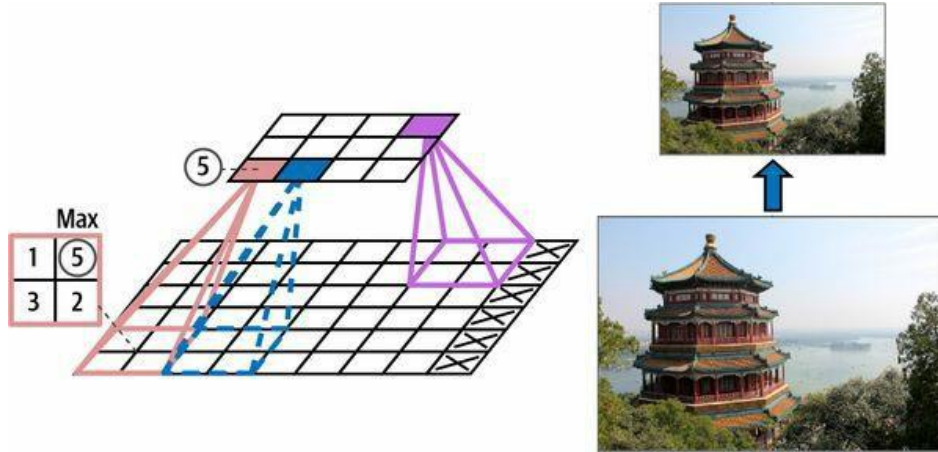
Əgər təlim yaddaş çatışmazlığı (out-of-memory) xətası ilə dağılırsa, mini-batch ölçüsünü azaltmağa cəhd edə bilərsiniz. Alternativ olaraq, stride istifadə edərək ölçünü azaltmağa, bir neçə qatı silməyə, 32-bitlik float əvəzinə 16-bitlik float istifadə etməyə, ya da CNN-i çoxsaylı qurğular arasında paylaşdırmağa (bunu necə edəcəyinizi Fəsil 19-da görəcəksiniz) cəhd edə bilərsiniz.

İndi gəlin CNN-lərin ikinci ümumi qurucu blokuna baxaq: pooling layer.

Pooling Layer-lər

Convolutional layer-lərin necə işlədiyini anladıqdan sonra pooling layer-ləri başa düşmək kifayət qədər asandır. Onların məqsədi hesablama yükünü, yaddaş istifadəsini və parametrlərin sayını (beləliklə overfitting riskini məhdudlaşdıraraq) azaltmaq üçün giriş şəklini *subsample* etmək (yəni kiçiltmək)dir.

Convolutional layer-lərdə olduğu kimi, pooling layer-dəki hər neyron əvvəlki qatda kiçik düzbucaqlı receptive field içində yerləşən məhdud sayda neyronun çıxışlarına bağlıdır. Əvvəlki kimi onun ölçüsünü, stride-ı və padding tipini təyin etməlisiniz. Lakin pooling neyronunun heç bir çəkisi (weights) yoxdur; etdiyi tək şey girişləri max və ya mean kimi aqreqasiya funksiyasından istifadə edərək toplamaqdır. Şəkil 14-9 ən geniş yayılmış pooling layer tipi olan *max pooling layer*-i göstərir. Bu nümunədə⁹ 2×2 pooling kernel-dən, 2 stride-dən və padding-siz istifadə edirik. Hər receptive field-də yalnız max giriş dəyəri növbəti qata keçir, qalan girişlər isə atılır. Məsələn, Şəkil 14-9-dakı aşağı sol receptive field-də giriş dəyərləri 1, 5, 3, 2-dir, ona görə də yalnız max dəyər, 5, növbəti qata ötürülür. 2 stride-a görə çıxış şəkli giriş şəklinin hündürlüyünün yarısına və eninin yarısına malikdir (padding istifadə etmədiyimiz üçün aşağıya yuvarlaqlaşdırılır).



Şəkil 14-9. Max pooling layer (2×2 pooling kernel, stride 2, padding yoxdur)

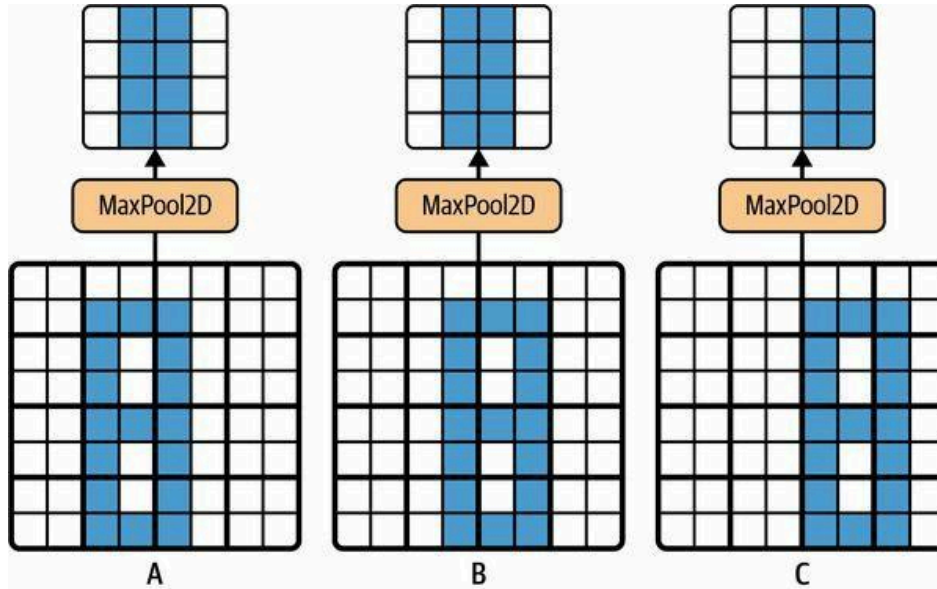
QEYD

Pooling layer adətən hər giriş kanalında müstəqil işləyir, ona görə də çıxış dərinliyi (yəni kanalların sayı) giriş dərinliyi ilə eynidir.

⁹İndiyə qədər müzakirə etdiyimiz digər kernel-lərin çəkili (weights) var idi, lakin pooling kernel-lərinin yoxdur: onlar sadəcə vəziyyətsiz (stateless) sürüşən pəncərələrdir.

Hesablamaları, yaddaş istifadəsini və parametrlərin sayını azaltmaqdan başqa, max pooling layer həm də kiçik sürüşmələrə (translations) qarşı müəyyən səviyyədə invariantlıq (invariance) gətirir (Şəkil 14-10-da göstərildiyi kimi). Burada parlaq piksellərin qaranlıq piksellərdən daha aşağı dəyərə malik olduğunu fərz edirik və 2×2 kernel və stride 2 ilə max pooling layer-dən keçən üç şəkli (A, B, C) nəzərdən keçiririk. B və C şəkilləri A şəkli ilə eynidir, lakin müvafiq olaraq bir və iki piksel sağa sürüşdürüldü. Gördüyünüz kimi, A və B şəkilləri üçün max pooling layer-in çıxışları eynidir. Translation invariance budur. C şəkli üçün çıxış fərqlidir: bir piksel sağa sürüşüb (lakin yenə də 50% invariantlıq var). CNN-də hər bir neçə qatdan bir max pooling layer əlavə etməklə daha böyük miqyasda müəyyən səviyyədə translation invariance əldə etmək mümkündür. Üstəlik, max pooling az miqdarda fırlanma invariantlığı (rotational invariance) və cüzi miqyas invariantlığı (scale invariance) təklif edir. Belə invariantlıq (məhdud olsa belə) proqnozun bu detallardan asılı olmamalı olduğu hallarda — məsələn təsnifat (classification) tapşırıqlarında — faydalı ola bilər.

Lakin max pooling-in bəzi mənfi tərəfləri də var. O, aydın şəkildə çox dağıcıdır: hətta kiçik 2×2 kernel və 2 stride ilə belə çıxış hər iki istiqamətdə iki dəfə kiçik olacaq (deməli onun sahəsi dörd dəfə kiçik olacaq), sadəcə giriş dəyərlərinin 75%-ni ataraq. Və bəzi tətbiqlərdə invariantlıq arzuolunmazdır. Semantic segmentation-ı götürün (hər pikseli aid olduğu obyektə görə təsnif etmə tapşırığı, ki bunu bu fəsildə bir az sonra araşdıracağıq): aydındır ki, əgər giriş şəkli bir piksel sağa sürüşdürülsə, çıxış da bir piksel sağa sürüşməlidir. Bu halda məqsəd invariantlıq deyil, *equivariance*-dir: girişlərdə kiçik dəyişiklik çıxışda müvafiq kiçik dəyişikliyə gətirib çıxarmalıdır.



Şəkil 14-10. Kiçik sürüşmələrə (translations) qarşı invariantlıq (invariance)

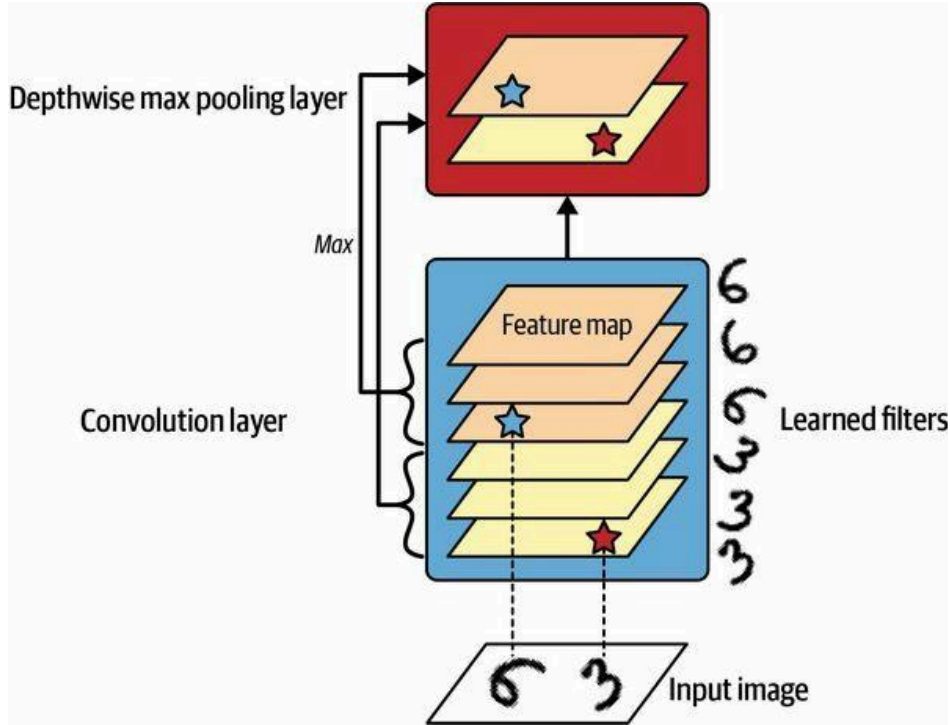
Pooling Layer-ləri Keras ilə Tətbiq Etmək

Aşağıdakı kod 2×2 kernel istifadə edərək MaxPooling2D layer-i (alias MaxPool2D) yaradır. Stride-lar susmaya görə kernel ölçüsünə bərabər olur, ona görə də bu qat 2 stride-dan istifadə edir (üfüqi və şaquli). Susmaya görə o, "valid" padding-dən (yəni heç bir padding olmadan) istifadə edir:


```
max_pool = tf.keras.layers.MaxPool2D(pool_size=2)
```

Average pooling layer yaratmaq üçün sadəcə MaxPool2D əvəzinə AveragePooling2D (alias AvgPool2D) istifadə edin. Gözlədiyiniz kimi, o, max pooling layer kimi işləyir, yalnız max əvəzinə mean hesablayır. Average pooling layer-lər əvvəllər çox populyar idi, lakin indi insanlar əsasən max pooling layer-lərdən istifadə edir, çünki onlar adətən daha yaxşı performans göstərir. Bu, təəccüblü görünə bilər, çünki mean hesablamaq adətən max hesablamaqdan daha az məlumat itirir. Lakin digər tərəfdən max pooling yalnız ən güclü xüsusiyyətləri saxlayır, bütün mənasız olanları ataraq, ona görə də növbəti qatlar işləmək üçün daha təmiz bir signal alır. Üstəlik, max pooling average pooling-dən daha güclü translation invariance təklif edir və cüzi olaraq daha az hesablama tələb edir.

Qeyd edin ki, max pooling və average pooling məkan ölçüləri əvəzinə dərinlik ölçüsü (depth dimension) boyunca da yerinə yetirilə bilər, baxmayaraq ki bu, o qədər də geniş yayılmayıb. Bu, CNN-ə müxtəlif xüsusiyyətlərə qarşı invariant olmağı öyrənməyə imkan verə bilər. Məsələn, o, eyni nümunənin müxtəlif fırlanmalarını aşkarlayan bir neçə filter öyrənə bilər (məsələn, əlyazma rəqəmləri; bax Şəkil 14-11) və depthwise max pooling layer çıxışın fırlanmadan asılı olmayaraq eyni olmasını təmin edərdi. CNN eyni şəkildə hər şeyə qarşı invariant olmağı öyrənə bilər: qalınlıq, parlaqlıq, əyilmə (skew), rəng və s.



Şəkil 14-11. Depthwise max pooling CNN-ə invariant olmağı öyrənməyə kömək edə bilər (bu halda fırlanmaya - rotation - görə)

Keras depthwise max pooling layer-i daxil etmir, lakin bunun üçün xüsusi (custom) qat tətbiq etmək o qədər də çətin deyil:

```
class DepthPool(tf.keras.layers.Layer):
    def __init__(self, pool_size=2, **kwargs):
        super().__init__(**kwargs)
        self.pool_size = pool_size

    def call(self, inputs):
        shape = tf.shape(inputs) # shape[-1] is the number of channels
        groups = shape[-1] // self.pool_size # number of channel groups
        new_shape = tf.concat([shape[:-1], [groups, self.pool_size]], axis=0)
        return tf.reduce_max(tf.reshape(inputs, new_shape), axis=-1)
```

Bu qat öz girişlərini kanalları istənilən ölçüdə (pool_size) qruplara bölmək üçün yenidən formalaşdırır (reshape), sonra hər qrupun max-ını hesablamaq üçün `tf.reduce_max()` istifadə edir. Bu tətbiq stride-in pool ölçüsünə bərabər olduğunu fərz edir ki, bu da ümumiyyətlə istədiyiniz şeydir. Alternativ olaraq, TensorFlow-un `tf.nn.max_pool()` əməliyyatından istifadə edib onu Keras modelində işlətmək üçün Lambda layer-ə bükə bilərsiniz, lakin təəssüf ki, bu əməliyyat depthwise pooling-i GPU üçün deyil, yalnız CPU üçün tətbiq edir.

Müasir arxitekturalarda tez-tez görəcəyiniz son bir pooling layer tipi *global average pooling layer*-dir. O, çox fərqli işləyir: etdiyi tək şey hər bütöv feature map-in mean-ini hesablamaqdır (bu, giriş ilə eyni məkan ölçülərinə malik pooling kernel istifadə edən average pooling layer kimidir). Bu o deməkdir ki, o, sadəcə hər feature map və hər nümunə üçün bir ədəd çıxarır. Bu, əlbəttə son dərəcə dağıdıcıdır (feature map-dəki məlumatın çox hissəsi itir), lakin bu fəsildə bir az sonra görəcəyiniz kimi, çıxış qatından bir az əvvəl faydalı ola bilər. Belə bir qat yaratmaq üçün sadəcə `GlobalAveragePooling2D` sinfini (alias `GlobalAvgPool2D`) istifadə edin:

```
global_avg_pool = tf.keras.layers.GlobalAvgPool2D()
```

Bu, məkan ölçüləri (height və width) üzrə mean hesablayan aşağıdakı Lambda layer-ə ekvivalentdir:

```
global_avg_pool = tf.keras.layers.Lambda(
    lambda X: tf.reduce_mean(X, axis=[1, 2]))
```

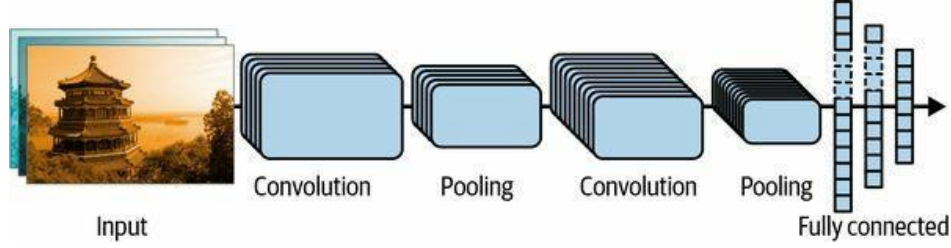
Məsələn, bu qatı giriş şəkillərinə tətbiq etsək, hər şəkil üçün qırmızı, yaşıl və mavinin mean intensivliyini alırıq:

```
>>> global_avg_pool(images)
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[0.64338624, 0.5971759 , 0.5824972 ],
       [0.76306933, 0.26011038, 0.10849128]], dtype=float32)>
```

İndi convolutional neural network yaratmaq üçün bütün qurucu blokları bilərsiniz. Gəlin onları necə yığacağımızı görək.

CNN Arxitekturaları

Tipik CNN arxitekturaları bir neçə convolutional layer-i (hər biri adətən ReLU layer-i ilə müşayiət olunur), sonra bir pooling layer-i, sonra daha bir neçə convolutional layer-i (+ReLU), sonra daha bir pooling layer-i və s. yığır. Şəkil şəbəkə boyunca irəlilədikcə getdikcə kiçilir, lakin convolutional layer-lər sayəsində adətən getdikcə daha dərin olur (yəni daha çox feature map ilə) (bax Şəkil 14-12). Yığının ən üstündə bir neçə fully connected layer-dən (+ReLU) ibarət adi feedforward neural network əlavə olunur və son qat proqnozu çıxarır (məsələn, təxmin olunan sınıf ehtimallarını çıxaran softmax layer).



Şəkil 14-12. Tipik CNN arxitekturası

İPUCU

Geniş yayılmış bir səhv həddən artıq böyük convolution kernel-ləri istifadə etməkdir. Məsələn, 5×5 kernel-li convolutional layer əvəzinə 3×3 kernel-li iki qat yığın: bu, daha az parametrlə istifadə ediləcək və daha az hesablama tələb ediləcək, həm də adətən daha yaxşı performans göstərəcək. Bir istisna birinci convolutional layer üçündür: o, adətən böyük kernel-ə (məsələn, 5×5) malik ola bilər, adətən 2 və ya daha çox stride ilə. Bu, şəklın məkan ölçüsünü çox məlumat itirmədən azaldacaq və giriş şəklının ümumiyyətlə yalnız üç kanalı olduğu üçün çox bahalı olmayacaq.

Fashion MNIST datasetini (Fəsil 10-da təqdim olunub) həll etmək üçün əsas bir CNN-i belə tətbiq edə bilərsiniz:

```
from functools import partial

DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, padding="same",
                        activation="relu", kernel_initializer="he_normal")
model = tf.keras.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(units=128, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=64, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=10, activation="softmax")
])
```

Gəlin bu kodu nəzərdən keçirək:

- `functools.partial()` funksiyasından (Fəsil 11-də təqdim olunub) istifadə edərək `DefaultConv2D`-ni təyin edirik; o, `Conv2D` kimi işləyir, lakin fərqli susmaya görə arqumentlərle: kiçik kernel ölçüsü 3, "same" padding, ReLU activation function və ona uyğun He initializer.
- Sonra `Sequential` model yaradıırıq. Onun birinci qatı 64 kifayət qədər böyük (7×7) filter-li `DefaultConv2D`-dir. Giriş şəkilləri çox böyük olmadığı üçün o, susmaya görə 1 stride istifadə edir. O, həmçinin `input_shape=[28, 28, 1]` təyin edir, çünki şəkillər 28×28 pikseldir, tək rəng kanalı ilə (yəni grayscale). Fashion MNIST datasetini yüklədikdə hər şəklın bu formada olmasına əmin olun: kanal ölçüsünü əlavə etmək üçün `np.reshape()`

və ya `np.expand_dims()` istifadə etməyiniz lazım gələ bilər. Alternativ olaraq, modeldə birinci qat kimi Reshape layer istifadə edə bilərsiniz.

- Sonra susmaya görə 2 pool ölçüsü istifadə edən max pooling layer əlavə edirik ki, o, hər məkan ölçüsünü 2 faktoruna bölür.
- Sonra eyni quruluşu iki dəfə təkrarlayırıq: iki convolutional layer, ardınca max pooling layer. Daha böyük şəkillər üçün bu quruluşu daha bir neçə dəfə təkrarlaya bilərdik. Təkrarların sayı tənzimlədiyiniz bir hiperparametrdir.
- Diqqət edin ki, CNN boyunca çıxış qatına doğru qalxdıqca filter-lərin sayı iki dəfə artır (əvvəlcə 64, sonra 128, sonra 256): onun böyüməsi məntiqlidir, çünki aşağı səviyyəli xüsusiyyətlərin sayı çox vaxt kifayət qədər azdır (məsələn, kiçik dairələr, üfüqi xətlər), lakin onları yüksək səviyyəli xüsusiyyətlərə birləşdirməyin çoxlu fərqli yolu var. Hər pooling layer-dən sonra filter-lərin sayını iki dəfə artırmaq geniş yayılmış praktikadır: pooling layer hər məkan ölçüsünü 2 faktoruna böldüyü üçün, parametrlərin sayını, yaddaş istifadəsini və ya hesablama yükünü partlatmaq qorxusu olmadan növbəti qatda feature map-lərin sayını iki dəfə artırmağı bacara bilərik.
- Sonra iki gizli dense qatdan və bir dense çıxış qatından ibarət fully connected şəbəkə gəlir. 10 sinifli təsnifat tapşırığı olduğu üçün çıxış qatında 10 unit var və o, softmax activation function istifadə edir. Diqqət edin ki, birinci dense qatdan tam əvvəl girişləri düzləşdirməliyik (flatten), çünki o, hər nümunə üçün 1D xüsusiyyət massivi gözləyir. Həmçinin overfitting-i azaltmaq üçün hər biri 50% dropout rate ilə iki dropout layer əlavə edirik.

Əgər bu modeli "sparse_categorical_crossentropy" loss ilə kompilyasiya edib (compile) Fashion MNIST təlim dəstinə uyğunlaşdırsanız (fit), o, test dəstində 92%-dən çox dəqiqliyə (accuracy) çatmalıdır. Bu, ən qabaqcıl (state of the art) deyil, lakin kifayət qədər yaxşıdır və Fəsil 10-da dense şəbəkələrlə əldə etdiyimizdən aydın şəkildə xeyli yaxşıdır.

İllər ərzində bu fundamental arxitekturanın variantları işlənib hazırlanaraq sahədə heyranəmiz irəliləyişlərə gətirib çıxarıb. Bu irəliləyişin yaxşı bir göstəricisi ILSVRC ImageNet challenge kimi yarışlardakı xəta dərəcəsidir. Bu yarışda şəkil təsnifatı üçün top-five error rate — yəni sistemin top beş proqnozuna düzgün cavabın daxil olmadığı test şəkillərinin sayı — cəmi altı ildə 26%-dən çoxdan 2,3%-dən aşağıya düşdü. Şəkillər kifayət qədər böyükdür (məsələn, 256 piksel hündürlükdə) və 1.000 sinif var, bəziləri həqiqətən incədir (120 it cinsini fərqləndirməyə çalışın). Qalib girişlərin təkamülünə baxmaq CNN-lərin necə işlədiyini və dərin öyrənmədə (deep learning) tədqiqatın necə irəlilədiyini anlamağın yaxşı bir yoludur.

Əvvəlcə klassik LeNet-5 arxitekturasına (1998), sonra ILSVRC challenge-in bir neçə qalibinə baxacağıq: AlexNet (2012), GoogLeNet (2014), ResNet (2015) və SENet (2017). Yol boyu Xception, ResNeXt, DenseNet, MobileNet, CSPNet və EfficientNet daxil olmaqla daha bir neçə arxitektura da baxacağıq.

LeNet-5

LeNet-5 arxitekturası bəlkə də ən geniş tanınan CNN arxitekturasıdır. Əvvəl qeyd olunduğu kimi, o, 1998-ci ildə Yann LeCun tərəfindən yaradılıb və əlyazma rəqəm tanınması (MNIST) üçün geniş istifadə olunub. O, Cədvəl 14-1-də göstərilən qatlardan ibarətdir.

Cədvəl 14-1. LeNet-5 arxitekturası

Layer	Type	Maps	Size	Kernel size
Out	Fully connected	–	10	–
F6	Fully connected	–	84	–
C5	Convolution	120	1 × 1	5 × 5
S4	Avg pooling	16	5 × 5	2 × 2
C3	Convolution	16	10 × 10	5 × 5
S2	Avg pooling	6	14 × 14	2 × 2
C1	Convolution	6	28 × 28	5 × 5
In	Input	1	32 × 32	–

Gördüyünüz kimi, bu, bizim Fashion MNIST modelimizə kifayət qədər oxşayır: convolutional layer-lər və pooling layer-lərdən ibarət bir yığın, ardınca dense şəbəkə. Bəlkə də daha müasir təsnifat CNN-ləri ilə əsas fərq activation function-lardır: bu gün tanh əvəzinə ReLU və RBF əvəzinə softmax istifadə edərdik. Əhəmiyyəti o qədər də olmayan bir neçə başqa cüzi fərq də var idi, amma maraqlanırsınızsa, onlar bu fəslin notebook-unda <https://homl.info/colab3> ünvanında sadalanıb. Yann LeCun-un veb saytında da LeNet-5-in rəqəmləri təsnif etdiyini göstərən əla demolar var.¹⁰

AlexNet

AlexNet¹¹ CNN arxitekturası 2012 ILSVRC challenge-i böyük fərqlə qazandı: o, 17% top-five error rate-ə nail oldu, ikinci ən yaxşı rəqib isə yalnız 26%-ə nail oldu! AlexNet Alex Krizhevsky (adı buradan gəlir), Ilya Sutskever və Geoffrey Hinton tərəfindən hazırlanıb. O, LeNet-5-ə oxşayır, sadəcə çox daha böyük və dərinir və hər convolutional layer-in üstünə pooling layer yığmaq əvəzinə convolutional layer-ləri birbaşa bir-birinin üstünə yığan ilk arxitektura oldu. Cədvəl 14-2 bu arxitekturanı təqdim edir.

Cədvəl 14-2. AlexNet arxitekturası

Layer	Type	Maps	Size	Kernel size
Out	Fully connected	–	1,000	–
F10	Fully connected	–	4,096	–
F9	Fully connected	–	4,096	–
S8	Max pooling	256	6 × 6	3 × 3
C7	Convolution	256	13 × 13	3 × 3
C6	Convolution	384	13 × 13	3 × 3
C5	Convolution	384	13 × 13	3 × 3
S4	Max pooling	256	13 × 13	3 × 3
C3	Convolution	256	27 × 27	5 × 5
S2	Max pooling	96	27 × 27	3 × 3

¹⁰Yann LeCun et al., "Gradient-Based Learning Applied to Document Recognition", Proceedings of the IEEE 86, no. 11 (1998): 2278–2324.

¹¹Alex Krizhevsky et al., "ImageNet Classification with Deep Convolutional Neural Networks", Proceedings of the 25th International Conference on Neural Information Processing Systems 1 (2012): 1097–1105.

Layer	Type	Maps	Size	Kernel size
C1	Convolution	96	55 × 55	11 × 11
In	Input	3 (RGB)	227 × 227	–

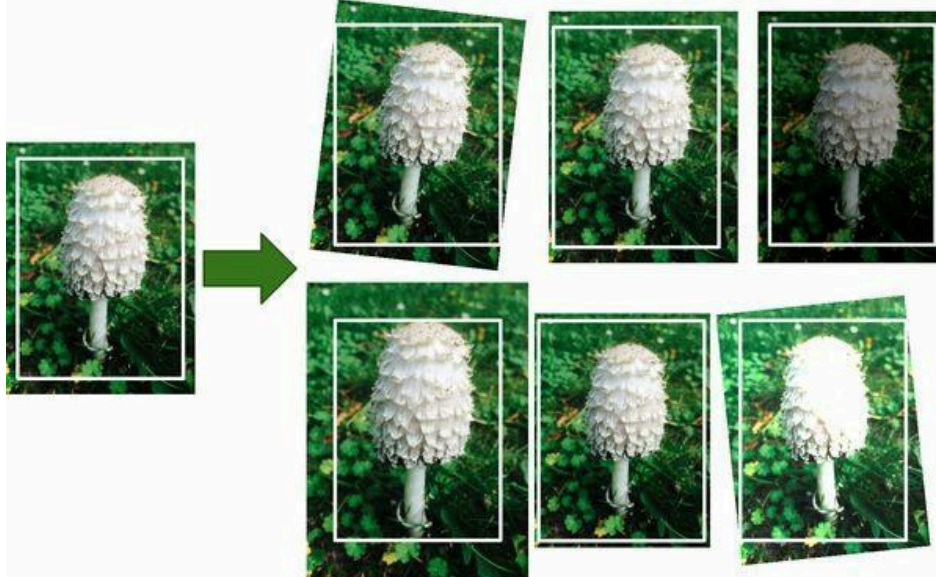
Overfitting-i azaltmaq üçün müəlliflər iki regularization üsulundan istifadə etdilər. Birincisi, təlim zamanı F9 və F10 qatlarının çıxışlarına 50% dropout rate ilə dropout (Fəsil 11-də təqdim olunub) tətbiq etdilər. İkincisi, təlim şəkillərini müxtəlif məsafələrlə təsadüfi sürüşdürərək, üfüqi çevirərək (flip) və işıqlandırma şəraitini dəyişərək data augmentation həyata keçirdilər.

DATA AUGMENTATION

Data augmentation hər təlim nümunəsinin çoxsaylı realistik variantlarını yaradaraq təlim dəstinin ölçüsünü süni şəkildə artırır. Bu, overfitting-i azaldır və bunu bir regularization üsulu edir. Yaradılan nümunələr mümkün qədər realistik olmalıdır: ideal halda, augment olunmuş təlim dəstindən bir şəkil verildikdə, insan onun augment olunub-olunmadığını ayırd edə bilməməlidir. Sadəcə ağ səs-küy (white noise) əlavə etmək kömək etməyəcək; dəyişikliklər öyrənilə bilən olmalıdır (white noise öyrənilə bilən deyil).

Məsələn, təlim dəstindəki hər şəkli müxtəlif miqdarlarda bir az sürüşdürə, fırlada və ölçüsünü dəyişə və alınan şəkilləri təlim dəstinə əlavə edə bilərsiniz (bax Şəkil 14-13). Bunun üçün Fəsil 13-də təqdim olunan Keras-ın data augmentation layer-lərindən (məsələn, RandomCrop, RandomRotation və s.) istifadə edə bilərsiniz. Bu, modeli şəkillərdəki obyektlərin mövqeyi, oriyentasiyası və ölçüsündəki dəyişikliklərə daha dözümlü olmağa məcbur edir. Müxtəlif işıqlandırma şəraitinə daha dözümlü model yaratmaq üçün eyni şəkildə müxtəlif kontrastlı çoxlu şəkil yarada bilərsiniz. Ümumiyyətlə, şəkilləri üfüqi də çevirə bilərsiniz (mətn və digər asimmetrik obyektlər istisna olmaqla). Bu çevrilmələri birləşdirməklə təlim dəstinizin ölçüsünü xeyli artırma bilərsiniz.

Data augmentation balanslaşmamış (unbalanced) datasetiniz olduqda da faydalıdır: daha az tezlikli siniflərin daha çox nümunəsini yaratmaq üçün ondan istifadə edə bilərsiniz. Buna *synthetic minority oversampling technique*, qısaca SMOTE deyilir.



Şəkil 14-13. Mövcud nümunələrdən yeni təlim nümunələrinin yaradılması

AlexNet həmçinin C1 və C3 qatlarının ReLU addımından dərhal sonra *local response normalization* (LRN) adlanan rəqabətli normallaşdırma addımı istifadə edir: ən güclü aktivləşmiş neyronlar qonşu feature map-lərdə eyni mövqedə yerləşən digər neyronları əngəlləyir (inhibit). Belə rəqabətli aktivləşmə bioloji neyronlarda müşahidə olunub. Bu, müxtəlif feature map-ləri ixtisaslaşmağa təşviq edir, onları bir-birindən uzaqlaşdırır və daha geniş xüsusiyyət spektrini araşdırmağa məcbur edir, nəticədə generalizasiyanı yaxşılaşdırır. Tənik 14-2 LRN-in necə tətbiq olunacağını göstərir.

Tənik 14-2. Local response normalization (LRN)

$$b_i = a_i \left(k + \alpha \sum_{j=j_{\text{low}}}^{j_{\text{high}}} a_j^2 \right)^{-\beta}$$

burada $j_{\text{high}} = \min(i + r/2, f_n - 1)$, $j_{\text{low}} = \max(0, i - r/2)$

Bu tənikdə:

- b_i hansısa u sətirində və v sütununda i feature map-ində yerləşən neyronun normallaşdırılmış çıxışıdır (qeyd edin ki, bu tənikdə yalnız bu sətirdə və sütunda yerləşən neyronları nəzərdən keçiririk, ona görə də u və v göstərilir).
- a_i həmin neyronun ReLU addımından sonra, lakin normallaşdırmadan əvvəlki aktivləşməsidir.
- k , α , β və r hiperparametrlərdir. k bias adlanır, r isə depth radius adlanır.
- f_n feature map-lərin sayıdır.

Məsələn, əgər $r = 2$ olarsa və bir neyronun güclü aktivləşməsi varsa, o, dərhal öz üstündə və altında yerləşən feature map-lərdəki neyronların aktivləşməsini əngəlləyəcək.

AlexNet-də hiperparametrlər belə təyin olunub: $r = 5$, $\alpha = 0.0001$, $\beta = 0.75$ və $k = 2$. Bu addımı `tf.nn.local_response_normalization()` funksiyasından istifadə edərək tətbiq edə bilərsiniz (əgər Keras modelində işlətmək istəyirsinizsə, onu Lambda layer-ə bükə bilərsiniz).

AlexNet-in *ZF Net* adlı bir variantı Matthew Zeiler və Rob Fergus tərəfindən hazırlanıb və 2013 ILSVRC challenge-i qazanıb.¹² O, əslində bir neçə tənzimlənmiş hiperparametrlə (feature map-lərin sayı, kernel ölçüsü, stride və s.) AlexNet-dir.

GoogLeNet

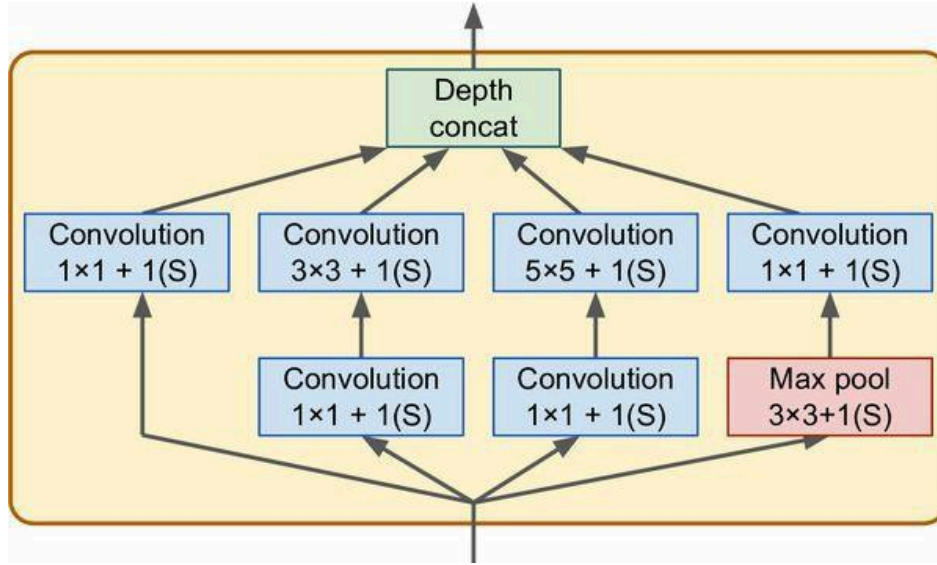
GoogLeNet arxitekturası Google Research-dən Christian Szegedy və başqaları tərəfindən hazırlanıb¹³ və top-five error rate-i 7%-dən aşağıya itələyərək ILSVRC 2014 challenge-i qazanıb. Bu əla performans böyük dərəcədə şəbəkənin əvvəlki CNN-lərdən çox daha dərin olması (Şəkil 14-15-də görəcəyiniz kimi) faktından gəldi. Bu, *inception module*-lar¹⁴ adlanan alt-şəbəkələr sayəsində mümkün oldu; onlar GoogLeNet-ə parametrlərdən əvvəlki arxitekturalardan çox daha səmərəli istifadə etməyə imkan verir: GoogLeNet-in əslində AlexNet-dən 10 dəfə az parametri var (60 milyon əvəzinə təxminən 6 milyon).

¹²Matthew D. Zeiler and Rob Fergus, "Visualizing and Understanding Convolutional Networks", Proceedings of the European Conference on Computer Vision (2014): 818–833.

¹³Christian Szegedy et al., "Going Deeper with Convolutions", Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2015): 1–9.

¹⁴2010-cu il "Inception" filmində personajlar getdikcə yuxuların çoxsaylı qatlarına daha dərin enirlər; bu modulların adı buradan gəlir.

Şəkil 14-14 bir inception module-un arxitekturasını göstərir. “ $3 \times 3 + 1(S)$ ” qeydi qatın 3×3 kernel, stride 1 və "same" padding istifadə etdiyini bildirir. Giriş signalı əvvəlcə paralel olaraq dörd fərqli qata verilir. Bütün convolutional layer-lər ReLU activation function istifadə edir. Diqqət edin ki, yuxarıdakı convolutional layer-lər fərqli kernel ölçüləri (1×1 , 3×3 və 5×5) istifadə edir ki, bu da onlara müxtəlif miqyaslarda nümunələri tutmağa imkan verir. Həmçinin diqqət edin ki, hər bir qat 1 stride və "same" padding istifadə edir (hətta max pooling layer də), ona görə də onların çıxışlarının hamısının hündürlüyü və eni girişləri ilə eynidir. Bu, son depth concatenation layer-də bütün çıxışları dərinlik ölçüsü boyunca birləşdirməyi (yəni dörd yuxarı convolutional layer-in feature map-lərini yığmağı) mümkün edir. Bunu Keras-ın Concatenate layer-i ilə, susmaya görə axis=-1 istifadə edərək tətbiq etmək olar.



Şəkil 14-14. Inception module

Niyə inception module-larda 1×1 kernel-li convolutional layer-lərin olması maraqlandırır bilər. Şübhəsiz ki, bu qatlar heç bir xüsusiyyət tuta bilməz, çünki onlar eyni anda yalnız bir pikselə baxır, elə deyilmi? Əslində bu qatlar üç məqsəddə xidmət edir:

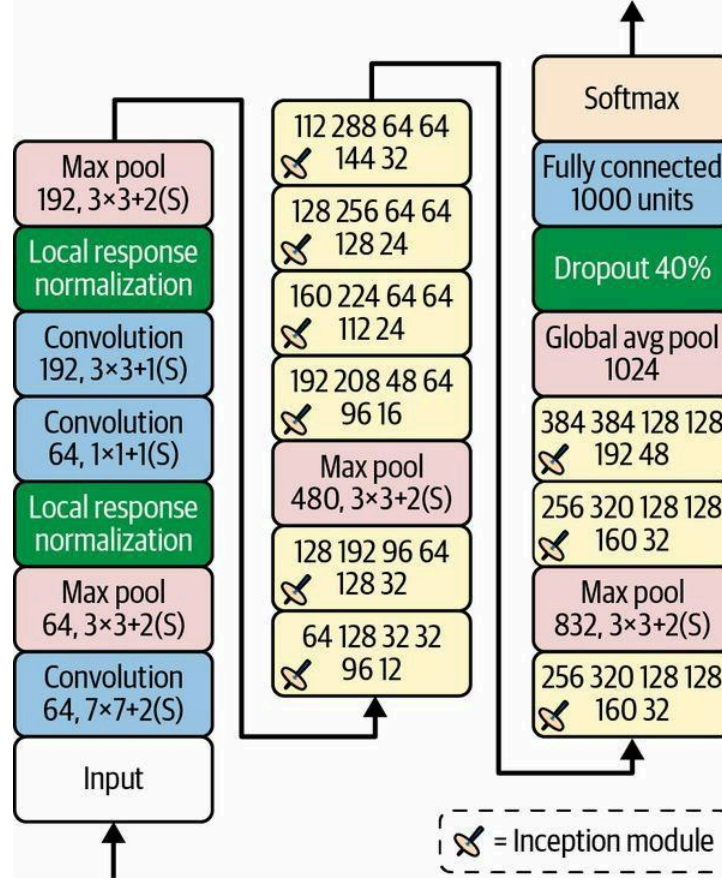
- Məkan nümunələrini (spatial patterns) tuta bilməsələr də, onlar dərinlik ölçüsü boyunca (yəni kanallar arası) nümunələri tuta bilər.
- Onlar girişlərindən daha az feature map çıxarmaq üçün konfigurasiya olunub, ona görə də *bottleneck layer* kimi xidmət edir, yəni ölçünü azaldır. Bu, hesablama xərcini və parametrlərin sayını azaldır, təlimi sürətləndirir və generalizasiyanı yaxşılaşdırır.
- Hər bir convolutional layer cütü ($[1 \times 1, 3 \times 3]$ və $[1 \times 1, 5 \times 5]$) bir tək güclü convolutional layer kimi davranır, daha mürəkkəb nümunələri tutmağa qadirdir. Convolutional layer şəkil boyunca dense layer-i sürüşdürməyə (hər yerdə yalnız kiçik bir receptive field-ə baxaraq) ekvivalentdir və bu convolutional layer cütləri şəkil boyunca ikiqatlı neural network-ləri sürüşdürməyə ekvivalentdir.

Qısacası, bütün inception module-u müxtəlif miqyaslarda mürəkkəb nümunələri tutan feature map-lər çıxara bilən, gücləndirilmiş bir convolutional layer kimi düşünə bilərsiniz.

İndi GoogLeNet CNN-in arxitekturasına baxaq (bax Şəkil 14-15). Hər convolutional layer və hər pooling layer tərəfindən çıxarılan feature map-lərin sayı kernel ölçüsündən əvvəl göstərilir. Arxitektura o qədər dərinidir ki, üç sütunda təmsil olunmalıdır, lakin GoogLeNet əslində doqquz inception module (fırlanan toplanı olan qutular) daxil olmaqla bir hündür yığındır. Inception module-lardakı altı rəqəm moduldakı hər convolutional layer tərəfindən çıxarılan feature map-lərin sayını təmsil edir (Şəkil 14-14-dəki ilə eyni ardıcılıqla). Diqqət edin ki, bütün convolutional layer-lər ReLU activation function istifadə edir.

Gəlin bu şəbəkəni nəzərdən keçirək:

- İlk iki qat şəklın hündürlüyünü və enini 4-ə bölür (deməli sahəsi 16-ya bölünür), hesablama yükünü azaltmaq üçün. Birinci qat böyük kernel ölçüsü, 7×7 istifadə edir ki, məlumatın çoxu qorunsun. Sonra local response normalization layer əvvəlki qatların müxtəlif xüsusiyyət spektrini öyrənməsini təmin edir (əvvəl müzakirə olunduğu kimi).
- İki convolutional layer gəlir, birincisi bottleneck layer kimi davranır. Qeyd olunduğu kimi, bu cütü bir tək daha ağıllı convolutional layer kimi düşünə bilərsiniz.
- Yenə də local response normalization layer əvvəlki qatların müxtəlif nümunələri tutmasını təmin edir.
- Sonra max pooling layer şəklın hündürlüyünü və enini 2-yə bölür, yenə hesablamaları sürətləndirmək üçün.
- Sonra CNN-in onurğası gəlir: ölçünü azaltmaq və şəbəkəni sürətləndirmək üçün bir neçə max pooling layer-lə aralıq verilmiş doqquz inception module-dan ibarət hündür yığın.
- Sonra global average pooling layer hər feature map-in mean-ini çıxarır: bu, qalan istənilən məkan məlumatını atır ki, bu da yaxşıdır, çünki o nöqtədə çox məkan məlumatı qalmayıb. Həqiqətən, GoogLeNet giriş şəkillərinin adətən 224×224 piksel olması gözlənilir, ona görə də hər biri hündürlüyü və enini 2-yə bölən 5 max pooling layer-dən sonra feature map-lər 7×7 -yə enir. Üstəlik, bu, lokalizasiya deyil, təsnifat tapşırığıdır, ona görə də obyektin harada olması əhəmiyyət kəsb etmir. Bu qatın gətirdiyi ölçü azalması sayəsində CNN-in üstündə bir neçə fully connected layer-ə (AlexNet-də olduğu kimi) ehtiyac yoxdur və bu, şəbəkədəki parametrlərin sayını xeyli azaldır və overfitting riskini məhdudlaşdırır.
- Son qatlar özünü izah edir: regularization üçün dropout, sonra 1.000 unit-li fully connected layer (1.000 sinif olduğu üçün) və təxmin olunan sinif ehtimallarını çıxarmaq üçün softmax activation function.



Şəkil 14-15. GoogLeNet arxitekturası

Orijinal GoogLeNet arxitekturasına üçüncü və altıncı inception module-ların üstünə qoşulmuş iki köməkçi təsnifatçı (auxiliary classifier) daxil idi. Onların hər ikisi bir average pooling layer-dən, bir convolutional layer-dən, iki fully connected layer-dən və bir softmax activation layer-dən ibarət idi. Təlim zamanı onların loss-u (70% azaldılmış) ümumi loss-a əlavə olunurdu. Məqsəd vanishing gradients problemi ilə mübarizə aparmaq və şəbəkəni regularize etmək idi, lakin sonradan göstərildi ki, onların təsiri nisbətən cüzi idi.

Sonradan Google tədqiqatçıları tərəfindən GoogLeNet arxitekturasının bir neçə variantı, o cümlədən bir az fərqli inception module-lardan istifadə edən Inception-v3 və Inception-v4 təklif olundu ki, bunlar daha yaxşı performansla nail oldu.

VGGNet

ILSVRC 2014 challenge-də ikinci yer VGGNet¹⁵ oldu; Oksford Universitetinin Visual Geometry Group (VGG) tədqiqat laboratoriyasından Karen Simonyan və Andrew Zisserman çox sadə və klassik bir arxitektura hazırladılar; onun 2 və ya 3 convolutional layer-i və bir pooling layer-i, sonra yenə 2 və ya 3 convolutional layer-i və bir pooling layer-i və s. (VGG variantından asılı olaraq ümumilikdə 16 və ya 19 convolutional layer-ə çataraq), üstəgəl 2 gizli qatdan və çıxış

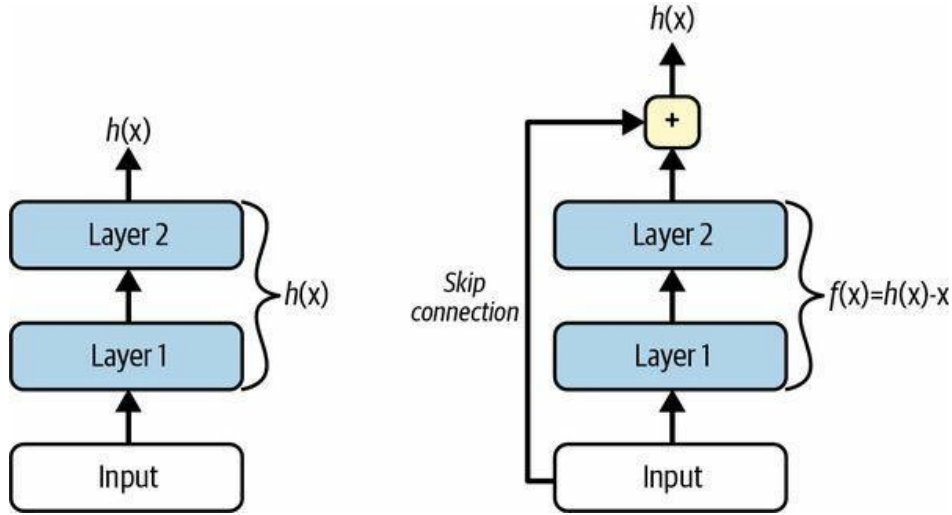
¹⁵Karen Simonyan and Andrew Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition", arXiv preprint arXiv:1409.1556 (2014).

qatından ibarət son dense şəbəkəsi var idi. O, kiçik 3×3 filter-lərdən istifadə edirdi, lakin onlardan çox sayda var idi.

ResNet

Kaiming He və başqaları 3,6%-dən aşağı heyrtəmiz top-five error rate verən Residual Network (ResNet)¹⁶ istifadə edərək ILSVRC 2015 challenge-i qazandılar. Qalib variant 152 qatdan ibarət son dərəcə dərin CNN istifadə etdi (digər variantların 34, 50 və 101 qatı var idi). Bu, ümumi tendensiyanı təsdiqlədi: computer vision modelləri getdikcə daha dərin, getdikcə daha az parametrlili olurdu. Belə dərin şəbəkəni təlim etməyin açarı *skip connection*-lardan (həmçinin *shortcut connection* adlanır) istifadə etməkdir: bir qata daxil olan signal həm də yığında daha yuxarıda yerləşən bir qatın çıxışına əlavə olunur. Gəlin bunun niyə faydalı olduğunu görək.

Neural network-ü təlim edərkən məqsəd onu hədəf funksiyanı $h(x)$ modelləşdirməyə vadar etməkdir. Əgər giriş x -i şəbəkənin çıxışına əlavə etsəniz (yəni skip connection əlavə etsəniz), onda şəbəkə $h(x)$ əvəzinə $f(x) = h(x) - x$ modelləşdirməyə məcbur olacaq. Buna *residual learning* deyilir (bax Şəkil 14-16).



Şəkil 14-16. Residual learning

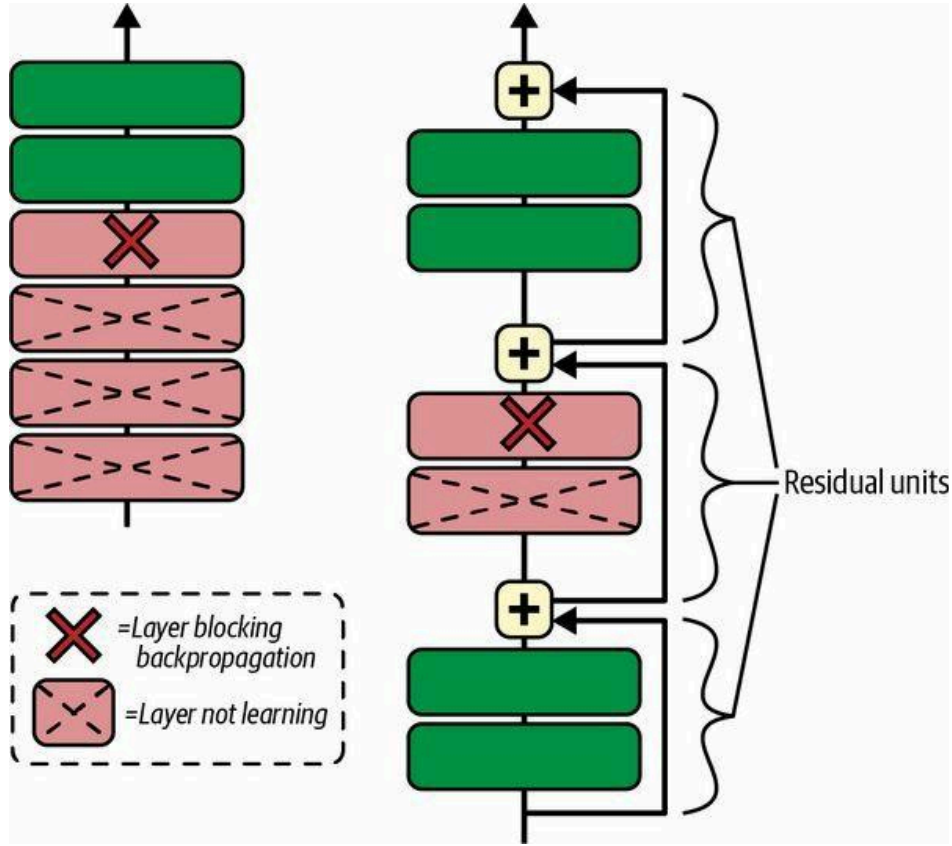
Adi neural network-ü ilkinləşdirdikdə onun çəkilişi sıfıra yaxındır, ona görə də şəbəkə sadəcə sıfıra yaxın dəyərlər çıxarır. Əgər skip connection əlavə etsəniz, alınan şəbəkə sadəcə öz girişlərinin sürətini çıxarır; başqa sözlə, o, əvvəlcə identity function modelləşdirir. Əgər hədəf funksiya identity function-a kifayət qədər yaxındırsa (ki çox vaxt belə olur), bu, təlimi xeyli sürətləndirəcək.

Üstəlik, çoxlu skip connection əlavə etsəniz, bir neçə qat hələ öyrənməyə başlamamış olsa belə şəbəkə irəliləyişə başlaya bilər (bax Şəkil 14-17). Skip connection-lar sayəsində signal bütün şəbəkə boyunca asanlıqla yol tapa bilər. Dərin residual network-ü *residual unit*-lərin (RU) yığını kimi düşünmək olar; burada hər residual unit skip connection-lu kiçik bir neural network-dür.

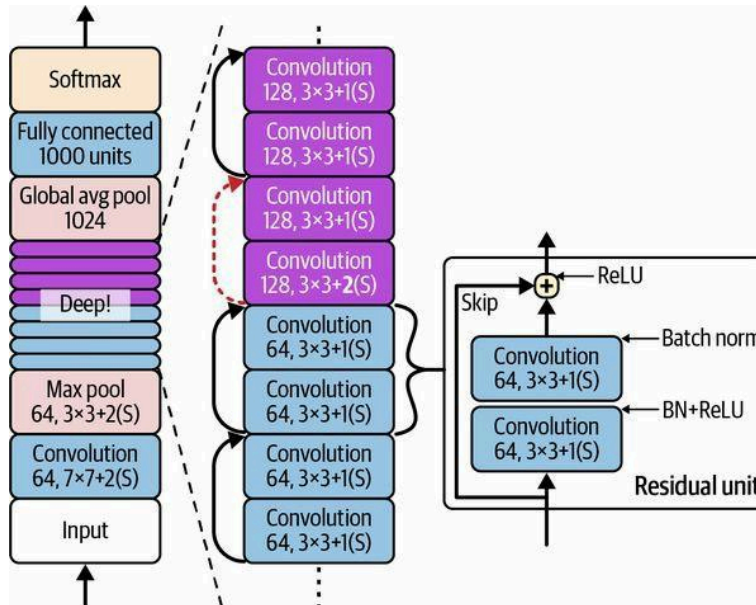
İndi ResNet-in arxitekturasına baxaq (bax Şəkil 14-18). O, təəccüblü dərəcədə sadədir. O, tam olaraq GoogLeNet kimi başlayır və bitir (dropout layer istisna olmaqla), aralarında isə sadəcə

¹⁶Kaiming He et al., "Deep Residual Learning for Image Recognition", arXiv preprint arXiv:1512.03385 (2015).

residual unit-lərin çox dərin yığını var. Hər residual unit iki convolutional layer-dən (və heç bir pooling layer yox!), batch normalization (BN) və ReLU activation ilə, 3×3 kernel istifadə edərək və məkan ölçülərini qoruyaraq (stride 1, "same" padding) ibarətdir.

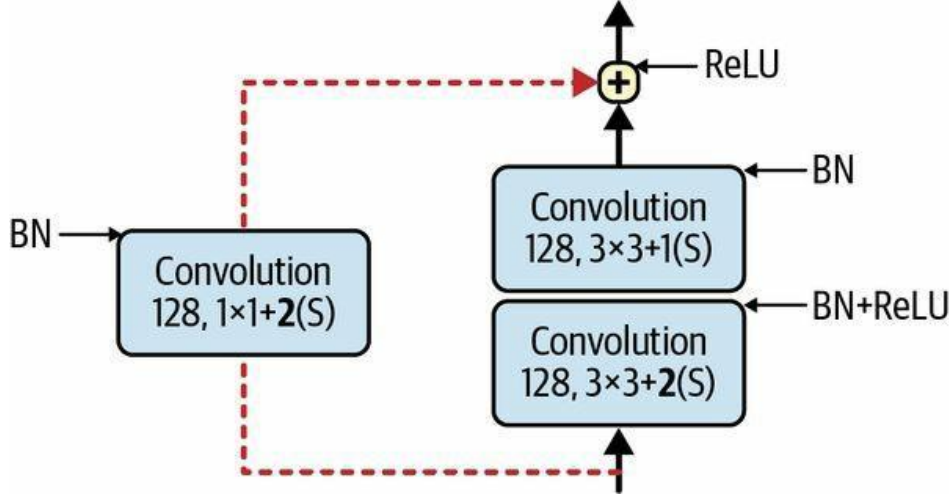


Şəkil 14-17. Adi dərin neural network (solda) və dərin residual network (sağda)



Şəkil 14-18. ResNet arxitekturası

Diqqət edin ki, feature map-lərin sayı hər bir neçə residual unit-dən bir iki dəfə artır, eyni zamanda onların hündürlüyü və eni yarıya bölünür (stride 2 ilə convolutional layer istifadə edərək). Bu baş verdikdə, girişləri birbaşa residual unit-in çıxışlarına əlavə etmək olmur, çünki onların eyni forması yoxdur (məsələn, bu problem Şəkil 14-18-də kəsik ox ilə təmsil olunan skip connection-a təsir edir). Bu problemi həll etmək üçün girişlər stride 2 və düzgün sayda çıxış feature map-i olan 1×1 convolutional layer-dən keçirilir (bax Şəkil 14-19).



Şəkil 14-19. Feature map ölçüsü və dərinliyi dəyişdikdə skip connection

Arxitekturanın müxtəlif sayda qatlı fərqli variantları var. ResNet-34 34 qatlı ResNet-dir (yalnız convolutional layer-ləri və fully connected layer-i sayaraq)¹⁷; o, 64 feature map çıxaran 3 RU, 128 map-li 4 RU, 256 map-li 6 RU və 512 map-li 3 RU saxlayır. Bu arxitekturanı bu fəsildə bir az sonra tətbiq edəcəyik.

QEYD

Google-un Inception-v4¹⁸ arxitekturası GoogLeNet və ResNet ideyalarını birləşdirdi və ImageNet təsnifatında 3%-ə yaxın top-five error rate-ə nail oldu.

Bundan daha dərin ResNet-lər, məsələn ResNet-152, bir az fərqli residual unit-lər istifadə edir. Məsələn 256 feature map-li iki 3×3 convolutional layer əvəzinə onlar üç convolutional layer istifadə edir: əvvəlcə cəmi 64 feature map-li 1×1 convolutional layer (4 x az) ki, bottleneck layer kimi davranır (artıq müzakirə olunduğu kimi), sonra 64 feature map-li 3×3 qat və nəhayət orijinal dərinliyi bərpa edən 256 feature map-li (4 dəfə 64) başqa bir 1×1 convolutional layer. ResNet-152 256 map çıxaran belə 3 RU, sonra 512 map-li 8 RU, 1.024 map-li tam 36 RU və nəhayət 2.048 map-li 3 RU saxlayır.

Xception

GoogLeNet arxitekturasının diqqətəlayiq başqa bir variantı var: *Xception*¹⁹ (Extreme Inception mənasını verir) 2016-cı ildə François Chollet (Keras-ın müəllifi) tərəfindən təklif olundu və

¹⁷Neural network təsvir edilərkən yalnız parametrləri olan layer-ləri saymaq adi praktikadır.

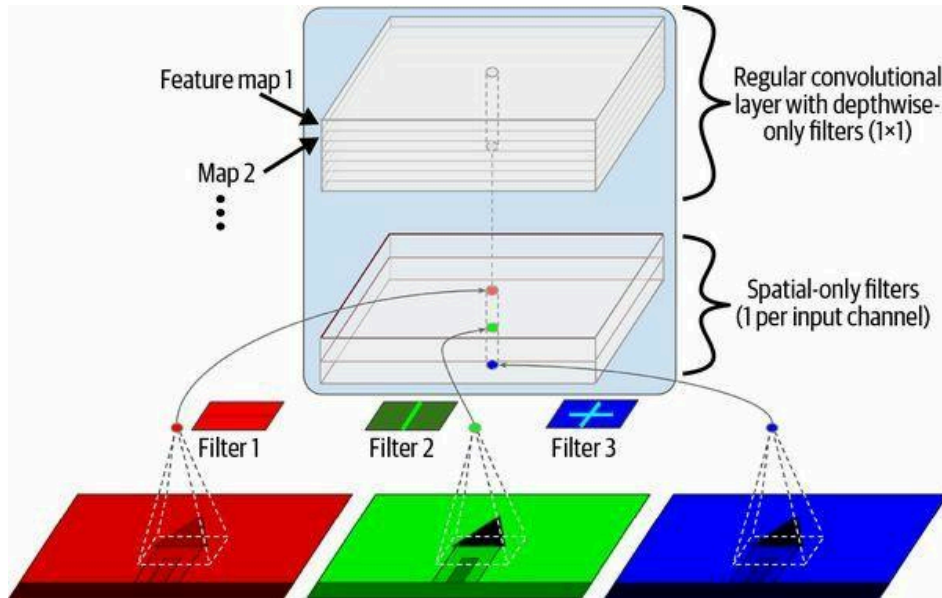
¹⁸Christian Szegedy et al., "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning", arXiv preprint arXiv:1602.07261 (2016).

¹⁹François Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions", arXiv preprint arXiv:1610.02357 (2016).

nəhəng bir vizual tapşırıqda (350 milyon şəkil və 17.000 sinif) Inception-v3-ü əhəmiyyətli dərəcədə üstələdi. Inception-v4 kimi, o, GoogLeNet və ResNet ideyalarını birləşdirir, lakin inception module-ları *depthwise separable convolution layer* (qısaca *separable convolution layer*²⁰) adlanan xüsusi bir qat tipi ilə əvəz edir. Bu qatlar əvvəllər bəzi CNN arxitekturalarında istifadə olunmuşdu, lakin Xception arxitekturasındakı qədər mərkəzi deyildi. Adi convolutional layer eyni anda həm məkan nümunələrini (məsələn, oval) həm də kanallar arasındakı nümunələri (məsələn, ağız + burun + gözlər = üz) tutmağa çalışan filter-lərdən istifadə etdiyi halda, separable convolutional layer məkan nümunələri ilə kanallar arasındakı nümunələrin ayrıca modelləşdirilə biləcəyinə dair güclü fərziyyə irəli sürür (bax Şəkil 14-20). Beləliklə, o, iki hissədən ibarətdir: birinci hissə hər giriş feature map-inə bir tək məkan filter-i tətbiq edir, ikinci hissə isə yalnız kanallar arasındakı nümunələri axtarır — bu, sadəcə 1×1 filter-li adi convolutional layer-dir.

Separable convolutional layer-lərin giriş kanalı başına yalnız bir məkan filter-i olduğu üçün, onları giriş qatı kimi çox az kanallı qatlardan sonra istifadə etməkdən çəkinməlisiniz (düzdür, Şəkil 14-20 məhz bunu təmsil edir, lakin bu, sadəcə illüstrasiya məqsədilədir). Bu səbəbdən Xception arxitekturası 2 adi convolutional layer-lə başlayır, lakin sonra arxitekturanın qalan hissəsi yalnız separable convolution-lardan (ümumilikdə 34) plus bir neçə max pooling layer-dən və adi son qatlardan (bir global average pooling layer və bir dense çıxış qatı) istifadə edir.

Xception-un niyə GoogLeNet-in variantı sayıldığını maraqlana bilərsiniz, axı onda heç bir inception module yoxdur. Yaxşı, əvvəl müzakirə olunduğu kimi, inception module 1×1 filter-li convolutional layer-lər saxlayır: bunlar yalnız kanallar arasındakı nümunələri axtarır. Lakin onların üstündə oturan convolutional layer-lər həm məkan, həm də kanallar arasındakı nümunələri axtaran adi convolutional layer-lərdir. Beləliklə, inception module-u adi convolutional layer (məkan və kanallar arasındakı nümunələri birgə nəzərdən keçirir) ilə separable convolutional layer (onları ayrıca nəzərdən keçirir) arasında bir aralıq kimi düşünə bilərsiniz. Praktikada görünür ki, separable convolutional layer-lər çox vaxt daha yaxşı performans göstərir.



²⁰Bu ad bəzən qeyri-müəyyən ola bilər, çünki məkanca ayrılabilir (spatially separable) convolution-lar da çox vaxt "separable convolution" adlanır.

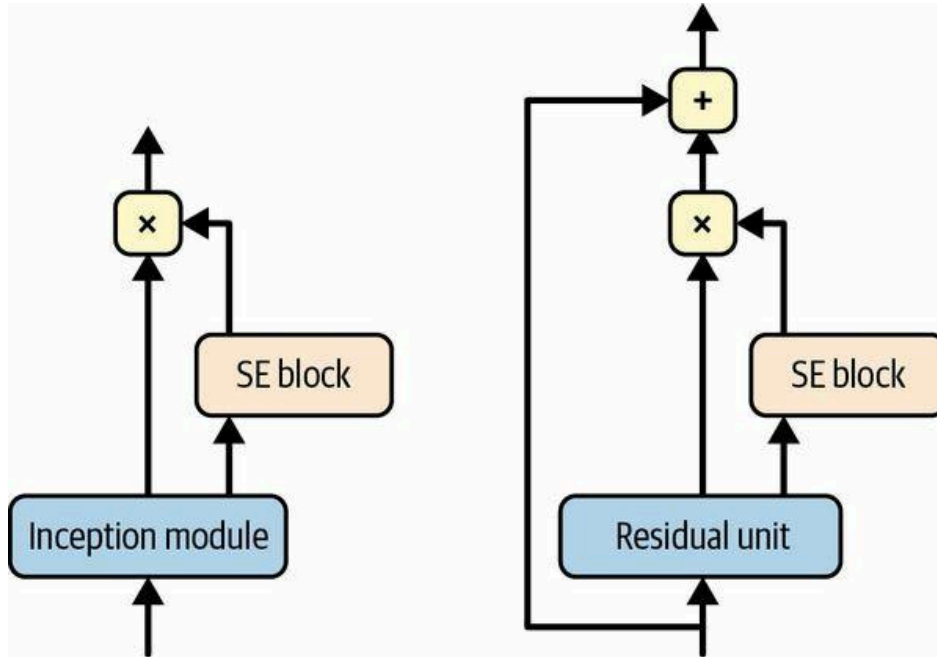
Şəkil 14-20. Depthwise separable convolutional layer

İPUCU

Separable convolutional layer-lər adi convolutional layer-lərdən daha az parametrlə, daha az yaddaş və daha az hesablama istifadə edir və çox vaxt daha yaxşı performans göstərir. Onları susmaya görə istifadə etməyi düşünün, az kanallı qatlardan (məsələn, giriş kanalı) sonra istisna olmaqla. Keras-da sadəcə Conv2D əvəzinə SeparableConv2D istifadə edin: bu, birbaşa əvəzedicidir (drop-in replacement). Keras həmçinin depthwise separable convolutional layer-in birinci hissəsini (yəni hər giriş feature map-inə bir məkan filter-i tətbiq etmə) tətbiq edən DepthwiseConv2D layer-i təklif edir.

SENet

ILSVRC 2017 challenge-də qalib arxitektura Squeeze-and-Excitation Network (SENet)²¹ idi. Bu arxitektura inception network-lər və ResNet-lər kimi mövcud arxitekturaları genişləndirir və onların performansını artırır. Bu, SENet-ə heyranamızı 2,25% top-five error rate ilə yarışı qazanmağa imkan verdi! Inception network-lər və ResNet-lərin genişləndirilmiş versiyaları müvafiq olaraq *SE-Inception* və *SE-ResNet* adlanırlar. Bu artım SENet-in orijinal arxitekturaadakı hər inception module-a və ya residual unit-ə *SE block* adlanan kiçik bir neural network əlavə etməsi faktından gəlir (Şəkil 14-21-də göstərildiyi kimi).

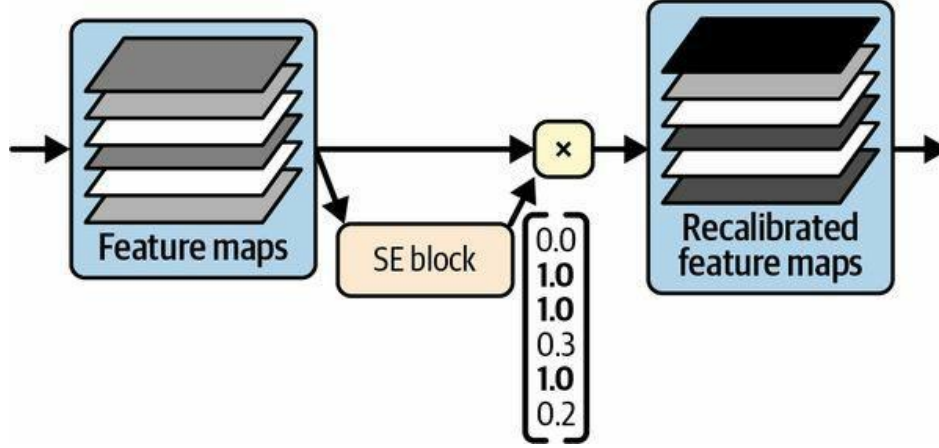


Şəkil 14-21. SE-Inception module (solda) və SE-ResNet unit (sağda)

SE block qoşulduğu unit-in çıxışını təhlil edir, yalnız dərinlik ölçüsünə diqqət yetirərək (heç bir məkan nümunəsi axtarmır) və adətən hansı xüsusiyyətlərin birlikdə ən aktiv olduğunu öyrənir. Sonra bu məlumatdan istifadə edərək feature map-ləri yenidən kalibrəyir (recalibrate), Şəkil 14-22-də göstərildiyi kimi. Məsələn, SE block ağız, burun və gözlərin adətən şəkillərdə birlikdə görüldüyünü öyrənə bilər: əgər ağız və burun görürsünüzsə, gözləri də görməyi gözləməlisiniz.

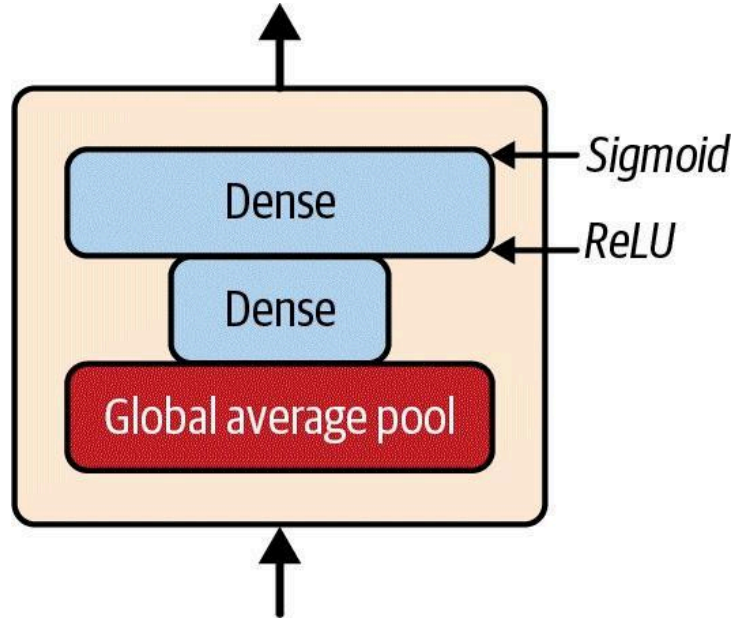
²¹Jie Hu et al., "Squeeze-and-Excitation Networks", Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2018): 7132–7141.

Beləliklə, əgər block ağz və burun feature map-lərində güclü aktivləşmə, lakin göz feature map-ində yalnız zəif aktivləşmə görürsə, o, göz feature map-ini gücləndirəcək (daha dəqiq desək, əlaqəsiz feature map-ləri azaldacaq). Əgər gözlər nəylə qarışdırılıbsa, bu feature map yenidən kalibrlənməsi qeyri-müəyyənliyi həll etməyə kömək edəcək.



Şəkil 14-22. SE block feature map rekalibrasiyasını həyata keçirir

SE block sadəcə üç qatdan ibarətdir: bir global average pooling layer, ReLU activation function istifadə edən bir gizli dense qat və sigmoid activation function istifadə edən bir dense çıxış qatı (bax Şəkil 14-23).



Şəkil 14-23. SE block arxitekturası

Əvvəlki kimi, global average pooling layer hər feature map üçün mean aktivləşməni hesablayır: məsələn, əgər onun girişi 256 feature map saxlayırsa, o, hər filter üçün ümumi reaksiya səviyyəsini təmsil edən 256 ədəd çıxaracaq. Növbəti qat “squeeze”-in baş verdiyi yerdir: bu qatın 256-dan əhəmiyyətli dərəcədə az neyronu var — adətən feature map-lərin sayından 16 dəfə az (məsələn, 16 neyron) — ona görə də 256 ədəd kiçik bir vektora (məsələn, 16 ölçü)

sıxılır. Bu, xüsusiyyət reaksiyalarının paylanması aşağı ölçülü vektor təsviridir (yeni *embedding*). Bu bottleneck addımı SE block-u xüsusiyyət kombinasiyalarının ümumi təsvirini öyrənməyə məcbur edir (bu prinsipi Fəsil 17-də autoencoder-ləri müzakirə edərkən yenidən fəaliyyətdə görəəcəyik). Nəhayət, çıxış qatı embedding-i götürür və hər feature map üçün bir ədəd (məsələn, 256) saxlayan, hər biri 0 ilə 1 arasında olan bir rekalibrasiya vektoru çıxarır. Sonra feature map-lər bu rekalibrasiya vektoruna vurulur, beləliklə əlaqəsiz xüsusiyyətlər (aşağı rekalibrasiya balı ilə) miqyasca kiçildilir, müvafiq xüsusiyyətlər isə (1-ə yaxın rekalibrasiya balı ilə) toxunulmaz qalır.

Digər Diqqətəlayiq Arxitekturalar

Araşdırmaq üçün bir çox başqa CNN arxitekturası var. Budur ən diqqətəlayiq olanlardan bəzilərinin qısa icmalı:

ResNeXt

ResNeXt²² ResNet-dəki residual unit-ləri yaxşılaşdırır. Ən yaxşı ResNet modellərindəki residual unit-lər hər biri sadəcə 3 convolutional layer saxladığı halda, ResNeXt residual unit-ləri hər biri 3 convolutional layer-li çoxsaylı paralel yığından (məsələn, 32 yığın) ibarətdir. Lakin hər yığındakı ilk iki qat yalnız bir neçə filter (məsələn, cəmi dörd) istifadə edir, ona görə də ümumi parametrlərin sayı ResNet-dəki ilə eyni qalır. Sonra bütün yığınların çıxışları toplanır və nəticə (skip connection ilə birlikdə) növbəti residual unit-ə ötürülür.

DenseNet

DenseNet²³ bir neçə dense block-dan ibarətdir, hər biri bir neçə sıx bağlı (densely connected) convolutional layer-dən ibarətdir. Bu arxitektura nisbətən az parametrlə istifadə edərək əla dəqiqliyə nail oldu. "Sıx bağlı" nə deməkdir? Hər qatın çıxışı eyni block daxilində ondan sonrakı hər qata giriş kimi verilir. Məsələn, block-dakı 4-cü qat həmin block-dakı 1, 2 və 3-cü qatların çıxışlarının dərinlik üzrə birləşməsinə (depthwise concatenation) giriş kimi qəbul edir. Dense block-lar bir neçə transition layer ilə ayrılır.

MobileNet

MobileNet-lər²⁴ yüngül və sürətli olmaq üçün hazırlanmış sadələşdirilmiş modellərdir ki, bu da onları mobil və veb tətbiqlərdə populyar edir. Onlar Xception kimi depthwise separable convolutional layer-lərə əsaslanır. Müəlliflər bir az dəqiqliyi daha sürətli və daha kiçik modellərə dəyişərək bir neçə variant təklif etdilər.

CSPNet

Cross Stage Partial Network (CSPNet)²⁵ DenseNet-ə bənzəyir, lakin hər dense block-un girişinin bir hissəsi block-dan keçmədən birbaşa həmin block-un çıxışına birləşdirilir.

EfficientNet

²²Saining Xie et al., "Aggregated Residual Transformations for Deep Neural Networks", arXiv preprint arXiv:1611.05431 (2016).

²³Gao Huang et al., "Densely Connected Convolutional Networks", arXiv preprint arXiv:1608.06993 (2016).

²⁴Andrew G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications", arXiv preprint arXiv:1704.04861 (2017).

²⁵Chien-Yao Wang et al., "CSPNet: A New Backbone That Can Enhance Learning Capability of CNN", arXiv preprint arXiv:1911.11929 (2019).

EfficientNet²⁶ təəribəsiz ki, bu siyahıdakı ən vacib modeldir. Müəlliflər istənilən CNN-i dərinliyini (qatların sayı), enini (qat başına filter-lərin sayı) və ayırdetməsini (input şəklində ölçüsü) prinsipial şəkildə birgə artıraraq səmərəli miqyaslandırmaq üçün bir metod təklif etdilər. Buna *compound scaling* deyilir. Onlar ImageNet-in kiçildilmiş versiyası (daha kiçik və daha az şəkillə) üçün yaxşı bir arxitektura tapmaq üçün neural architecture search istifadə etdilər, sonra bu arxitekturanın getdikcə daha böyük versiyalarını yaratmaq üçün compound scaling istifadə etdilər. EfficientNet modelləri çıxanda onlar bütün hesablama büdcələrində mövcud bütün modelləri kəskin şəkildə üstələdilər və bu gün də ən yaxşı modellər sırasında qalırlar.

EfficientNet-in compound scaling metodunu başa düşmək CNN-ləri daha dərinləndirən anlamaq üçün faydalıdır, xüsusilə nə vaxtsa CNN arxitekturasını miqyaslandırmaq lazım gələrsə. O, ϕ ilə qeyd olunan hesablama büdcəsinin loqarifmik ölçüsünə əsaslanır: əgər hesablama büdcəniz iki dəfə artarsa, ϕ 1 vahid artır. Başqa sözlə, təlim üçün mövcud floating-point əməliyyatlarının sayı 2^ϕ ilə mütənasibdir. CNN arxitekturanızın dərinliyi, eni və ayırdetməsi müvafiq olaraq α^ϕ , β^ϕ və γ^ϕ kimi miqyaslanmalıdır. α , β və γ faktorları 1-dən böyük olmalı və $\alpha \times \beta^2 \times \gamma^2$ 2-yə yaxın olmalıdır. Bu faktorlar üçün optimal dəyərlər CNN-in arxitekturasından asılıdır. EfficientNet arxitekturası üçün optimal dəyərləri tapmaq üçün müəlliflər kiçik bir baseline modeldən (EfficientNetB0) başladılar, $\phi = 1$ sabit saxladılar və sadəcə grid search apardılar: onlar $\alpha = 1.2$, $\beta = 1.1$ və $\gamma = 1.1$ tapdılar. Sonra bu faktorlardan istifadə edərək getdikcə daha böyük ϕ dəyərləri üçün EfficientNetB1-dən EfficientNetB7-yə qədər adlanan bir neçə daha böyük arxitektura yaratdılar.

Düzgün CNN Arxitekturasını Seçmək

Bu qədər çox CNN arxitekturası ilə layihənin üçün hansının ən yaxşı olduğunu necə seçirsiniz? Bu, sizin üçün ən vacib olandan asılıdır: Dəqiqlik (Accuracy)? Model ölçüsü (məsələn, mobil qurğuya yerləşdirmə üçün)? CPU-da çıxarış (inference) sürəti? GPU-da? Cədvəl 14-3 model ölçüsünə görə sıralanmış, Keras-da hazırda mövcud ən yaxşı pretrained modelləri sadalayır (onları necə istifadə edəcəyinizi bu fəsildə bir az sonra görəcəksiniz). Tam siyahını <https://keras.io/api/applications> ünvanında tapa bilərsiniz. Hər model üçün cədvəl istifadə ediləcək Keras sinif adını (tf.keras.applications paketində), modelin MB ilə ölçüsünü, ImageNet datasetində top-1 və top-5 validation dəqiqliyini, parametrlərin sayını (milyonla) və 32 şəkillik batch-lərlə kifayət qədər güclü aparatda²⁷ CPU və GPU-da ms ilə çıxarış vaxtını göstərir. Hər sütun üçün ən yaxşı dəyər vurğulanıb. Gördüyünüz kimi, daha böyük modellər ümumiyyətlə daha dəqiqdir, lakin həmişə yox; məsələn, EfficientNetB2 InceptionV3-ü həm ölçüdə, həm də dəqiqlikdə üstələyir. InceptionV3-ü siyahıda saxladım, çünki o, CPU-da EfficientNetB2-dən təxminən iki dəfə sürətlidir. Eyni şəkildə, InceptionResNetV2 CPU-da sürətlidir, ResNet50V2 və ResNet101V2 isə GPU-da heyranamiz dərəcədə sürətlidir.

Cədvəl 14-3. Keras-da mövcud pretrained modellər

Class name	Size (MB)	Top-1 acc	Top-5 acc	Params
MobileNetV2	14	71.3%	90.1%	3.5M

²⁶Mingxing Tan and Quoc V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks", arXiv preprint arXiv:1905.11946 (2019).

²⁷92 nüvəli AMD EPYC CPU (IBPB ilə), 1,7 TB RAM və Nvidia Tesla A100 GPU.

Class name	Size (MB)	Top-1 acc	Top-5 acc	Params
MobileNet	16	70.4%	89.5%	4.3M
NASNetMobile	23	74.4%	91.9%	5.3M
EfficientNetB0	29	77.1%	93.3%	5.3M
EfficientNetB1	31	79.1%	94.4%	7.9M
EfficientNetB2	36	80.1%	94.9%	9.2M
EfficientNetB3	48	81.6%	95.7%	12.3M
EfficientNetB4	75	82.9%	96.4%	19.5M
InceptionV3	92	77.9%	93.7%	23.9M
ResNet50V2	98	76.0%	93.0%	25.6M
EfficientNetB5	118	83.6%	96.7%	30.6M
EfficientNetB6	166	84.0%	96.8%	43.3M
ResNet101V2	171	77.2%	93.8%	44.7M
InceptionResNetV2	215	80.3%	95.3%	55.9M
EfficientNetB7	256	84.3%	97.0%	66.7M

Ümid edirəm ki, əsas CNN arxitekturalarına bu dərin baxışdan həzz aldınız! İndi gəlin onlardan birini Keras-dan istifadə edərək necə tətbiq edəcəyimizi görək.

Keras ilə ResNet-34 CNN-in Tətbiqi

İndiyə qədər təsvir olunan əksər CNN arxitekturaları Keras-dan istifadə edərək kifayət qədər təbii şəkildə tətbiq oluna bilər (baxmayaraq ki, adətən, görəcəyiniz kimi, bunun əvəzinə pretrained bir şəbəkə yükləyərdir). Prosesi nümayiş etdirmək üçün gəlin Keras ilə sıfırdan bir ResNet-34 tətbiq edək. Əvvəlcə bir ResidualUnit layer-i yaradacağıq:

```

DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, strides=1,
                        padding="same", kernel_initializer="he_normal",
                        use_bias=False)

class ResidualUnit(tf.keras.layers.Layer):
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = tf.keras.activations.get(activation)
        self.main_layers = [
            DefaultConv2D(filters, strides=strides),
            tf.keras.layers.BatchNormalization(),
            self.activation,
            DefaultConv2D(filters),
            tf.keras.layers.BatchNormalization()
        ]
        self.skip_layers = []
        if strides > 1:
            self.skip_layers = [
                DefaultConv2D(filters, kernel_size=1, strides=strides),
                tf.keras.layers.BatchNormalization()
            ]

    def call(self, inputs):
        Z = inputs
        for layer in self.main_layers:
            Z = layer(Z)
        skip_Z = inputs
        for layer in self.skip_layers:
            skip_Z = layer(skip_Z)
        return self.activation(Z + skip_Z)

```

Gördüyünüz kimi, bu kod Şəkil 14-19-a kifayət qədər yaxından uyğun gəlir. Konstruktorda lazım olacaq bütün layer-ləri yaradırıq: əsas layer-lər (main layers) diaqramın sağ tərəfindəkilərdir, skip layer-lər isə sol tərəfdəkilərdir (yalnız stride 1-dən böyük olduqda lazımdır). Sonra call() metodunda girişləri əsas layer-lərdən və (varsa) skip layer-lərdən keçiririk, hər iki çıxışı toplayırıq və activation function-ı tətbiq edirik.

İndi Sequential model istifadə edərək ResNet-34 qura bilərik, çünki o, əslində sadəcə uzun bir layer ardıcılığıdır — artıq ResidualUnit sinfinə sahib olduğumuz üçün hər residual unit-i tək bir layer kimi qəbul edə bilərik. Kod Şəkil 14-18-ə yaxından uyğun gəlir:

```

model = tf.keras.Sequential([
    DefaultConv2D(64, kernel_size=7, strides=2, input_shape=[224, 224, 3]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.MaxPool2D(pool_size=3, strides=2, padding="same"),
])
prev_filters = 64
for filters in [64] * 3 + [128] * 4 + [256] * 6 + [512] * 3:
    strides = 1 if filters == prev_filters else 2
    model.add(ResidualUnit(filters, strides=strides))
    prev_filters = filters
model.add(tf.keras.layers.GlobalAvgPool2D())
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(10, activation="softmax"))

```

Bu kodda yeganə çətin hissə ResidualUnit layer-lərini modelə əlavə edən döngüdür: əvvəl izah edildiyi kimi, ilk 3 RU-nun 64 filter-i, sonrakı 4 RU-nun 128 filter-i və s. var. Hər iterasiyada filter-lərin sayı əvvəlki RU-dakı ilə eyni olduqda stride-ı 1-ə, əks halda 2-yə təyin etməliyik; sonra ResidualUnit-i əlavə edirik və nəhayət prev_filters-i yeniləyirik.

Heyrətamizdir ki, təxminən 40 sətir kodla 2015-ci il ILSVRC çağırışını qazanan modeli qura bilərik! Bu, həm ResNet modelinin zərifliyini, həm də Keras API-nin ifadəliliyini (expressiveness) nümayiş etdirir. Digər CNN arxitekturalarını tətbiq etmək bir az daha uzundur, lakin çox da çətin deyil. Lakin Keras bu arxitekturaların bir neçəsini özündə hazır təqdim edir, ona görə də niyə onların əvəzinə bunlardan istifadə etməyəək?

Keras-dan Pretrained Modellərdən İstifadə

Ümumiyyətlə, GoogLeNet və ya ResNet kimi standart modelləri əl ilə tətbiq etmək məcburiyyətində qalmayacaqsınız, çünki `tf.keras.applications` paketində pretrained şəbəkələr bir sətir kodla hazır mövcuddur.

Məsələn, ImageNet üzərində pretrained ResNet-50 modelini aşağıdakı kod sətri ilə yükləyə bilərsiniz:

```
model = tf.keras.applications.ResNet50(weights="imagenet")
```

Hamısı bu qədər! Bu, ResNet-50 modeli yaradacaq və ImageNet dataseti üzərində pretrained çəkiləri yükləyəcək. Ondan istifadə etmək üçün əvvəlcə şəkillərin düzgün ölçüdə olduğundan əmin olmalısınız. ResNet-50 modeli 224×224 piksellik şəkillər gözləyir (digər modellər başqa ölçülər gözləyə bilər, məsələn 299×299), ona görə də iki nümunə şəkli (hədəf en-boy nisbətinə — aspect ratio — kəsildikdən sonra) yenidən ölçüləndirmək (resize) üçün Keras-ın Resizing layer-indən (Fəsil 13-də təqdim olunub) istifadə edək:

```
images = load_sample_images()["images"]
images_resized = tf.keras.layers.Resizing(height=224, width=224,
                                          crop_to_aspect_ratio=True)(images)
```

Pretrained modellər şəkillərin müəyyən bir şəkildə ön emal olunduğunu (preprocessed) fərz edir. Bəzi hallarda onlar girişlərin 0-dan 1-ə və ya -1 -dən 1-ə qədər miqyaslandırılmasını gözləyə bilər və s. Hər model şəkillərinizi ön emal etmək üçün istifadə edə biləcəyiniz bir `preprocess_input()` funksiyası təqdim edir. Bu funksiyalar orijinal piksel dəyərlərinin 0-dan 255-ə qədər olduğunu fərz edir ki, bu da burada belədir:

```
inputs = tf.keras.applications.resnet50.preprocess_input(images_resized)
```

İndi proqnoz vermək üçün pretrained modeldən istifadə edə bilərik:

```
>>> Y_proba = model.predict(inputs)
>>> Y_proba.shape
(2, 1000)
```

Adətən olduğu kimi, çıxış `Y_proba` şəkil başına bir sətir və sinif başına bir sütun olan bir matrisdir (bu halda 1.000 sinif var). Hər sinfin adı və təxmin olunan ehtimalı da daxil olmaqla ən yaxşı K proqnozu göstərmək istəyirsinizsə, `decode_predictions()` funksiyasından istifadə edin. Hər şəkil üçün o, ən yaxşı K proqnozu ehtiva edən bir massiv qaytarır; hər proqnoz sinif identifikatorunu,²⁸ adını və müvafiq inam (confidence) balını ehtiva edən bir massiv kimi təmsil olunur:

²⁸ImageNet datasetində hər şəkil WordNet datasetindəki bir sözə uyğunlaşdırılır: sinif ID-si sadəcə bir WordNet ID-sidir.

```
top_K = tf.keras.applications.resnet50.decode_predictions(Y_proba, top=3)
for image_index in range(len(images)):
    print(f"Image #{image_index}")
    for class_id, name, y_proba in top_K[image_index]:
        print(f"  {class_id} - {name:12s} {y_proba:.2%}")
```

Çıxış belə görünür:

```
Image #0
n03877845 - palace      54.69%
n03781244 - monastery   24.72%
n02825657 - bell_cote   18.55%
Image #1
n04522168 - vase        32.66%
n11939491 - daisy       17.81%
n03530642 - honeycomb   12.06%
```

Düzgün siniflər palace və dahlia-dır, ona görə də model birinci şəkil üçün düzgün, ikinci üçün isə səhvdir. Lakin bunun səbəbi dahlia-nın 1.000 ImageNet sinfindən biri olmamasıdır. Bunu nəzərə alsaq, vase ağlabatan bir təxmindir (bəlkə gül bir vazadadır?), daisy də pis seçim deyil, çünki dahlia və daisy hər ikisi eyni Compositae fəsiləsindəndir.

Gördüyünüz kimi, pretrained modeldən istifadə edərək kifayət qədər yaxşı bir şəkil təsnifatçısı (image classifier) yaratmaq çox asandır. Cədvəl 14-3-də gördüyünüz kimi, tf.keras.applications-da yüngül və sürətli modellərdən tutmuş böyük və dəqiq modellərə qədər bir çox başqa vizual model mövcuddur.

Bəs ImageNet-in bir hissəsi olmayan şəkil sinifləri üçün bir şəkil təsnifatçısı istifadə etmək istəyirsinizsə? Bu halda, pretrained modellərdən transfer learning həyata keçirmək üçün istifadə edərək yenə də fayda görə bilərsiniz.

Transfer Learning üçün Pretrained Modellər

Bir şəkil təsnifatçısı qurmaq istəyirsinizsə, lakin onu sıfırdan öyrətmək üçün kifayət qədər məlumatınız yoxdursa, çox vaxt Fəsil 11-də müzakirə etdiyimiz kimi, pretrained modelin aşağı layer-lərini təkrar istifadə etmək yaxşı fikirdir. Məsələn, gəlin pretrained Xception modelini təkrar istifadə edərək gül şəkillərini təsnif etmək üçün bir model öyrədək. Əvvəlcə TensorFlow Datasets-dən (Fəsil 13-də təqdim olunub) istifadə edərək flowers datasetini yükləyəcəyik:

```
import tensorflow_datasets as tfds

dataset, info = tfds.load("tf_flowers", as_supervised=True, with_info=True)
dataset_size = info.splits["train"].num_examples # 3670
class_names = info.features["label"].names # ["dandelion", "daisy", ...]
n_classes = info.features["label"].num_classes # 5
```

Diqqət edin ki, with_info=True təyin edərək dataset haqqında məlumat ala bilərsiniz. Burada datasetin ölçüsünü və siniflərin adlarını alırıq. Təəssüf ki, yalnız bir "train" dataseti var, test set və ya validation set yoxdur, ona görə də training set-i bölməliyik. tfds.load()-u yenidən çağırıq, lakin bu dəfə datasetin ilk 10%-ni test üçün, sonrakı 15%-ni validation üçün və qalan 75%-ni training üçün götürək:

```
test_set_raw, valid_set_raw, train_set_raw = tfds.load(
    "tf_flowers",
    split=["train[:10%]", "train[10%:25%]", "train[25%:]"],
    as_supervised=True)
```


Hər üç dataset fərdi şəkillərdən ibarətdir. Onları batch-lərə bölməliyik, lakin əvvəlcə hamısının eyni ölçüdə olduğundan əmin olmalıyıq, əks halda batch-ləmə uğursuz olacaq. Bunun üçün Resizing layer-indən istifadə edə bilərik. Həmçinin şəkilləri Xception modeli üçün uyğun şəkildə ön emal etmək üçün `tf.keras.applications.xception.preprocess_input()` funksiyasını çağırmalıyıq. Nəhayət, training set-i qarışdıracaq (shuffle) və prefetching istifadə edəcəyik:

```
batch_size = 32
preprocess = tf.keras.Sequential([
    tf.keras.layers.Resizing(height=224, width=224, crop_to_aspect_ratio=True),
    tf.keras.layers.Lambda(tf.keras.applications.xception.preprocess_input)
])
train_set = train_set_raw.map(lambda X, y: (preprocess(X), y))
train_set = train_set.shuffle(1000, seed=42).batch(batch_size).prefetch(1)
valid_set = valid_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
test_set = test_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
```

İndi hər batch 32 şəkildən ibarətdir, hamısı 224×224 piksel, piksel dəyərləri -1 -dən 1 -ə qədər. Əla!

Dataset o qədər də böyük olmadığı üçün bir az data augmentation mütləq kömək edəcək. Gəlin son modelimizə yerləşdirəcəyimiz bir data augmentation modeli yaradaq. Təlim zamanı o, şəkilləri təsadüfi olaraq üfüqi çevirəcək (flip), onları bir az fırladacaq və kontrastı (contrast) tənzimləyəcək:

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip(mode="horizontal", seed=42),
    tf.keras.layers.RandomRotation(factor=0.05, seed=42),
    tf.keras.layers.RandomContrast(factor=0.2, seed=42)
])
```

İPUCU

`tf.keras.preprocessing.image.ImageDataGenerator` sinfi şəkilləri diskdən yükləməyi və müxtəlif yollarla augment etməyi asanlaşdırır: hər şəkli sürüşdürə, fırlada, yenidən miqyaslandır (rescale), üfüqi və ya şaquli çevirə, kəsə (shear) və ya ona istədiyiniz hər hansı çevrilmə funksiyasını tətbiq edə bilərsiniz. Bu, sadə layihələr üçün çox əlverişlidir. Lakin `tf.data` pipeline-ı o qədər də mürəkkəb deyil və ümumiyyətlə daha sürətlidir. Üstəlik, GPU-nuz varsa və ön emal və ya data augmentation layer-lərini modelinizin içinə daxil etsəniz, onlar təlim zamanı GPU sürətləndirməsindən faydalanacaq.

İndi ImageNet üzərində pretrained Xception modelini yükləyək. `include_top=False` təyin edərək şəbəkənin üst hissəsini istisna edirik. Bu, global average pooling layer-ini və dense çıxış layer-ini istisna edir. Sonra öz global average pooling layer-imizi əlavə edirik (ona base model-in çıxışını veririk), ardınca sinif başına bir vahidi olan, softmax activation function istifadə edən dense çıxış layer-i gəlir. Nəhayət, bütün bunları bir Keras Model-ə bükürük:

```
base_model = tf.keras.applications.xception.Xception(weights="imagenet",
                                                    include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
model = tf.keras.Model(inputs=base_model.input, outputs=output)
```

Fəsil 11-də izah edildiyi kimi, ən azı təlimin əvvəlində pretrained layer-lərin çəkirlərini dondurmaq (freeze) adətən yaxşı fikirdir:

```
for layer in base_model.layers:
    layer.trainable = False
```

XƏBƏRDARLIQ

Modelimiz `base_model` obyektinin özünü deyil, `base_model`in `layer`-lərini birbaşa istifadə etdiyi üçün `base_model.trainable=False` təyin etmək heç bir təsir göstərməzdi.

Nəhayət, modeli `compile` edib təlimə başlamaq bilərik:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=3)
```

XƏBƏRDARLIQ

Əgər Colab-da işləyirsinizsə, `runtime`-ın GPU istifadə etdiyindən əmin olun: `Runtime` → “Change runtime type” seçin, “Hardware accelerator” açılan menyusunda “GPU”-nu seçin, sonra `Save` düyməsinə klikləyin. Modeli GPU olmadan da öyrətmək mümkündür, lakin bu, dəhşətli dərəcədə yavaş olacaq (epoxa başına dəqiqələrlə, saniyələrlə deyil).

Modeli bir neçə epoxa öyrətdikdən sonra onun `validation` dəqiqliyi (`accuracy`) 80%-dən bir az çox olmalı və sonra yaxşılaşmağı dayandırmalıdır. Bu o deməkdir ki, üst `layer`-lər artıq kifayət qədər yaxşı öyrədilib və biz `base_model`in bəzi üst `layer`-lərini açmağa (`unfreeze`), sonra təlimi davam etdirməyə hazırıq. Məsələn, gəlin 56 və ondan yuxarı `layer`-ləri açaq (`layer` adlarını sadalasanız görəcəyiniz kimi, bu, 14-dən 7-ci `residual unit`-in başlanğıcıdır):

```
for layer in base_model.layers[56:]:
    layer.trainable = True
```

`Layer`-ləri dondurduğunuz və ya açdığınız zaman modeli yenidən `compile` etməyi unutmayın. Həmçinin `pretrained` çəkilərə zərər verməmək üçün daha aşağı `learning rate` istifadə etdiyinizdən əmin olun:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=10)
```

Bu model `test set`-də təxminən 92% dəqiqliyə çatmalıdır, cəmi bir neçə dəqiqəlik təlimlə (GPU ilə). Əgər `hyperparametrləri` tənzimləsəniz, `learning rate`-i aşağı salsanız və xeyli uzun müddət öyrətsəniz, 95%-dən 97%-ə qədər çata bilməlisiniz. Bununla, öz şəkilləriniz və sinifləriniz üzərində heyranəmiz şəkil təsnifatçıları öyrətməyə başlaya bilərsiniz! Lakin `computer vision` təkcə təsnifatdan ibarət deyil. Məsələn, gülün şəkildə harada olduğunu da bilmək istəyirsinizsə? Gəlin indi buna baxaq.

Təsnifat və Lokalizasiya (Classification and Localization)

Şəkildə bir obyektin lokalizasiyası (`localizing`) Fəsil 10-da müzakirə edildiyi kimi bir `regressiya` (`regression`) tapşırığı kimi ifadə oluna bilər: obyektin ətrafında bir `bounding box` proqnozlaşdırmaq üçün geniş yayılmış bir yanaşma obyektin mərkəzinin üfüqi və şaquli koordinatlarını, eləcə də onun hündürlüyünü və enini proqnozlaşdırmaqdır. Bu o deməkdir ki, proqnozlaşdırmaq üçün dörd rəqəmimiz var. Bu, modeldə çox dəyişiklik tələb etmir; sadəcə dörd vahidi olan ikinci bir `dense çıxış layer-i` əlavə etməliyik (adətən `global average pooling layer`-in üstündə) və o, `MSE loss` istifadə edərək öyrədilə bilər:

```

base_model = tf.keras.applications.xception.Xception(weights="imagenet",
                                                    include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
class_output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
loc_output = tf.keras.layers.Dense(4)(avg)
model = tf.keras.Model(inputs=base_model.input,
                      outputs=[class_output, loc_output])
model.compile(loss=["sparse_categorical_crossentropy", "mse"],
              loss_weights=[0.8, 0.2], # depends on what you care most about
              optimizer=optimizer, metrics=["accuracy"])

```

Lakin indi bir problemimiz var: flowers dataseti güllərin ətrafında bounding box-lara malik deyil. Ona görə də onları özümüz əlavə etməliyik. Bu, çox vaxt bir machine learning layihəsinin ən çətin və ən bahalı hissələrindən biridir: etiketləri (labels) əldə etmək. Düzgün alətlər axtarmağa vaxt sərf etmək yaxşı fikirdir. Şekilləri bounding box-larla annotasiya etmək üçün VGG Image Annotator, LabelImg, OpenLabeler və ya ImgLab kimi açıq mənbəli (open source) bir şəkil etikətləmə aləti, yaxud LabelBox və ya Supervisely kimi kommersiya aləti istifadə etmək istəyə bilərsiniz. Annotasiya ediləcək çox böyük sayda şəkliniz varsa, Amazon Mechanical Turk kimi crowdsourcing platformalarını da nəzərdən keçirə bilərsiniz. Lakin crowdsourcing platforması qurmaq, işçilərə göndiriləcək formanı hazırlamaq, onlara nəzarət etmək və istehsal etdikləri bounding box-ların keyfiyyətinin yaxşı olduğundan əmin olmaq xeyli işdir, ona görə də buna dəyər olduğundan əmin olun. Adriana Kovashka və başqaları computer vision-da crowdsourcing haqqında çox praktik bir məqalə yazıblar.²⁹ Crowdsourcing istifadə etməyi planlaşdırmırsanız belə, ona baxmağı tövsiyə edirəm. Etiketlənəcək yalnız bir neçə yüz və ya hətta bir-iki min şəkil varsa və bunu tez-tez etməyi planlaşdırmırsınızsa, onu özünüz etmək daha yaxşı ola bilər: düzgün alətlərlə bu, yalnız bir neçə gün çəkəcək və həm də datasetiniz və tapşırığınız haqqında daha yaxşı anlayış əldə edəcəksiniz.

İndi fərz edək ki, flowers datasetindəki hər şəkil üçün bounding box-ları əldə etmisiniz (hələlik şəkil başına tək bir bounding box olduğunu fərz edəcəyik). Sonra elementləri ön emal olunmuş şəkillərin, onların sinif etikətlərinin və bounding box-larının batch-ləri olan bir dataset yaratmalısınız. Hər element (images, (class_labels, bounding_boxes)) formasında bir tuple olmalıdır. Sonra modelinizi öyrətməyə hazırsınız!

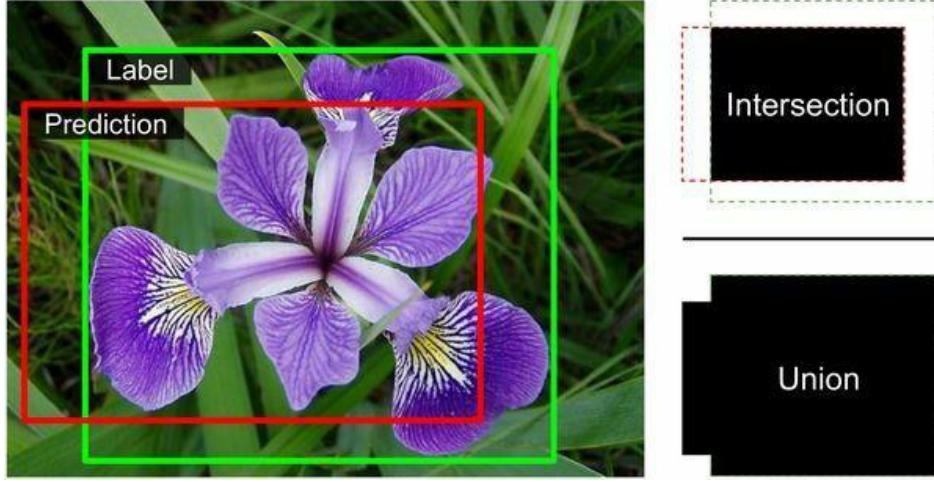
İPUCU

Bounding box-lar normallaşdırılmalıdır (normalized) ki, üfüqi və şaquli koordinatlar, eləcə də hündürlük və en hamısı 0-dan 1-ə qədər dəyişsin. Həmçinin hündürlük və enin özünü deyil, onların kvadrat kökünü proqnozlaşdırmaq geniş yayılıb: bu yolla böyük bir bounding box üçün 10 piksellə xəta kiçik bir bounding box üçün 10 piksellə xəta qədər cəzalandırılmayacaq.

MSE adətən modeli öyrətmək üçün cost function kimi kifayət qədər yaxşı işləyir, lakin o, modelin bounding box-ları nə dərəcədə yaxşı proqnozlaşdırma bildiyini qiymətləndirmək üçün yaxşı bir metrik deyil. Bunun üçün ən geniş yayılmış metrik *intersection over union* (IoU)-dur: proqnozlaşdırılan bounding box ilə hədəf bounding box arasındakı üst-üstə düşmə (overlap) sahəsinin onların birləşməsinin (union) sahəsinə bölünməsi (bax: Şəkil 14-24). Keras-da o, `tf.keras.metrics.MeanIoU` sinfi ilə tətbiq olunur.

²⁹Adriana Kovashka et al., "Crowdsourcing in Computer Vision", Foundations and Trends in Computer Graphics and Vision 10, no. 3 (2014): 177–243.

Tək bir obyektı təsnif etmək və lokalizasiya etmək yaxşıdır, lakin şəkillərdə birdən çox obyekt varsa (flowers datasetində tez-tez olduğu kimi) nə etməli?



Şəkil 14-24. Bounding box-lar üçün IoU metriği

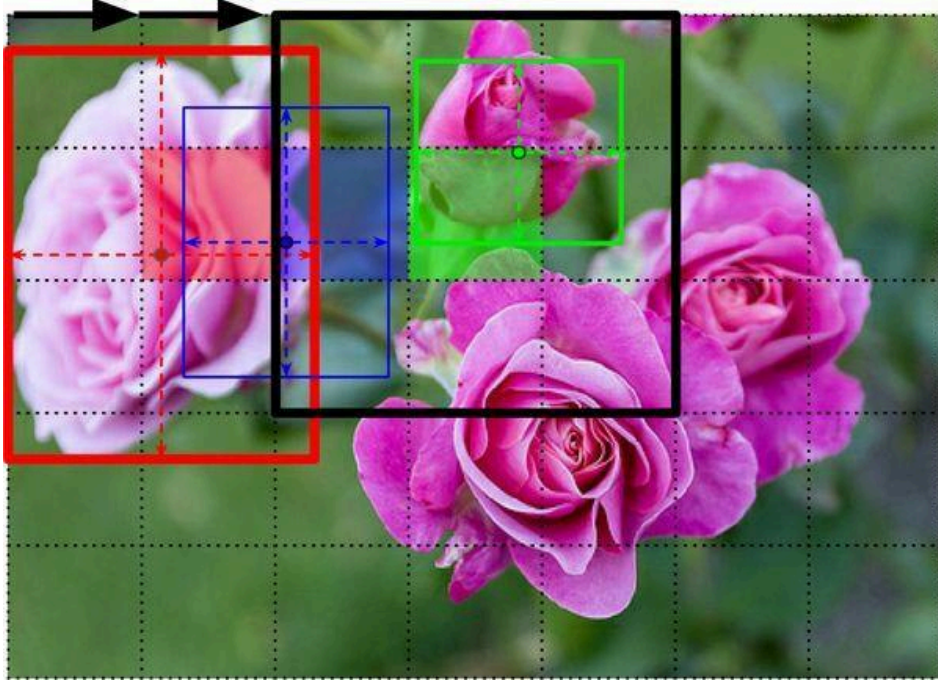
Object Detection

Bir şəkildə birdən çox obyektin təsnif edilməsi və lokalizasiyası tapşırığı *object detection* adlanır. Bir neçə il əvvələ qədər geniş yayılmış bir yanaşma şəkildə təxminən mərkəzləşdirilmiş tək bir obyektı təsnif etmək və lokalizasiya etmək üçün öyrədilmiş bir CNN götürmək, sonra bu CNN-i şəkil boyunca sürüşdürmək və hər addımda proqnozlar vermək idi. CNN adətən tək cə sinif ehtimallarını və bir bounding box-u deyil, həm də bir *objectness score* proqnozlaşdırmaq üçün öyrədilirdi: bu, şəklın həqiqətən mərkəzə yaxın bir obyekt ehtiva etməsinin təxmin olunan ehtimalıdır. Bu, ikili (binary) bir təsnifat çıxışıdır; o, sigmoid activation function istifadə edən və binary cross-entropy loss istifadə edərək öyrədilən tək vahidli bir dense çıxış layer-i tərəfindən istehsal oluna bilər.

QEYD

Objectness score əvəzinə bəzən bir “no-object” sinfi əlavə olunurdu, lakin ümumiyyətlə bu, o qədər də yaxşı işləmirdi: “Obyekt mövcuddurmu?” və “Hansı növ obyektidir?” sualları ən yaxşı ayrıca cavablandırılır.

Bu sürüşən-CNN (sliding-CNN) yanaşması Şəkil 14-25-də təsvir olunub. Bu nümunədə şəkil 5×7 şəbəkəyə (grid) bölünüb və biz bir CNN-in — qalın qara düzbucaqlının — bütün 3×3 bölgələr boyunca sürüşdüyünü və hər addımda proqnozlar verdiyini görürük.



Şəkil 14-25. CNN-i şəkil boyunca sürüşdürərək birdən çox obyektin aşkarlanması

Bu şəkildə CNN artıq bu 3×3 bölgələrdən üçü üçün proqnozlar verib:

- Yuxarı-sol 3×3 bölgəyə baxarkən (ikinci sətirdə və ikinci sütunda yerləşən qırmızı-çalarlı şəbəkə xanasında mərkəzləşmiş), o, ən soldakı qızılgülü aşkarladı. Diqqət edin ki, proqnozlaşdırılan bounding box bu 3×3 bölgənin sərhədini aşır. Bu, tamamilə normaldır: CNN qızılgülün aşağı hissəsini görə bilməsə də, onun harada ola biləcəyinə dair əlabatan bir təxmin verə bildi. O, həmçinin sinif ehtimallarını proqnozlaşdırdı və “rose” sinfinə yüksək ehtimal verdi. Nəhayət, o, kifayət qədər yüksək bir objectness score proqnozlaşdırdı, çünki bounding box-un mərkəzi mərkəzi şəbəkə xanasının içindədir (bu şəkildə objectness score bounding box-un qalınlığı ilə təmsil olunub).
- Bir şəbəkə xanası sağda növbəti 3×3 bölgəyə baxarkən (çalarlı mavi kvadratda mərkəzləşmiş), o, həmin bölgədə mərkəzləşmiş heç bir gül aşkarlamadı, ona görə də çox aşağı bir objectness score proqnozlaşdırdı; buna görə proqnozlaşdırılan bounding box və sinif ehtimalları təhlükəsiz şəkildə nəzərə alınmaya bilər. Onsuz da proqnozlaşdırılan bounding box-un yararsız olduğunu görə bilərsiniz.
- Nəhayət, yenə bir şəbəkə xanası sağda növbəti 3×3 bölgəyə baxarkən (çalarlı yaşıl xanada mərkəzləşmiş), o, yuxarıdakı qızılgülü aşkarladı, baxmayaraq ki, mükəmməl deyil: bu qızılgül bu bölgədə yaxşı mərkəzləşməyib, ona görə də proqnozlaşdırılan objectness score çox yüksək deyildi.

Təsəvvür edə bilərsiniz ki, CNN-i bütün şəkil boyunca sürüşdürmək sizə 3×5 şəbəkədə təşkil olunmuş ümumilikdə 15 proqnozlaşdırılmış bounding box verərdi, hər bounding box öz təxmin olunan sinif ehtimalları və objectness score-u ilə müşayiət olunur. Obyektlər müxtəlif ölçülərə malik ola bildiyi üçün, daha çox bounding box əldə etmək üçün CNN-i daha böyük 4×4 bölgələr boyunca da sürüşdürmək istəyə bilərsiniz.

Bu texnika kifayət qədər sadədir, lakin gördüyünüz kimi, o, çox vaxt eyni obyektə bir az fərqli mövqelərdə dəfələrlə aşkarlayacaq. Bütün lazımsız bounding box-lardan xilas olmaq üçün bir qədər son emal (postprocessing) lazımdır. Bunun üçün geniş yayılmış bir yanaşma *non-max suppression* adlanır. Bu, belə işləyir:

1. Əvvəlcə objectness score-u müəyyən bir həddən (threshold) aşağı olan bütün bounding box-lardan xilas olun: CNN həmin yerdə obyekt olmadığına inandığı üçün bounding box yararsızdır.
2. Qalan bounding box-lar arasında ən yüksək objectness score-a malik olanı tapın və onunla çox üst-üstə düşən (məsələn, IoU 60%-dən böyük) bütün digər qalan bounding box-lardan xilas olun. Məsələn, Şəkil 14-25-də maksimum objectness score-a malik bounding box ən soldakı qızılgülün üzərindəki qalın bounding box-dur. Bu eyni qızılgülə toxunan digər bounding box maksimum bounding box ilə çox üst-üstə düşür, ona görə də ondan xilas olacağıq (baxmayaraq ki, bu nümunədə o, onsuz da əvvəlki addımda silinmiş olardı).
3. Xilas olunacaq başqa bounding box qalmayana qədər 2-ci addımı təkrarlayın.

Object detection-a bu sadə yanaşma kifayət qədər yaxşı işləyir, lakin o, CNN-i dəfələrlə (bu nümunədə 15 dəfə) işlətməyi tələb edir, ona görə də kifayət qədər yavaşdır. Xoşbəxtlikdən, CNN-i bir şəkil boyunca sürüşdürmənin daha sürətli bir yolu var: *fully convolutional network* (FCN) istifadə etmək.

Fully Convolutional Network-lər

FCN-lərin ideyası ilk dəfə 2015-ci ildə Jonathan Long və başqalarının semantic segmentation (bir şəkildəki hər pikselin aid olduğu obyektin sinfinə görə təsnif edilməsi tapşırığı) üçün yazdığı bir məqalədə təqdim olunub.³⁰ Müəlliflər qeyd etdilər ki, CNN-in üstündəki dense layer-ləri convolutional layer-lərlə əvəz edə bilərsiniz. Bunu başa düşmək üçün bir nümunəyə baxaq: fərz edək ki, 100 feature map çıxaran bir convolutional layer-in üstündə 200 neyronlu bir dense layer oturur, hər feature map 7×7 ölçüsündədir (bu, feature map ölçüsüdür, kernel ölçüsü deyil). Hər neyron convolutional layer-dən bütün $100 \times 7 \times 7$ aktivasiyaların çəkili cəmini (üstəgəl bias term) hesablayacaq. İndi dense layer-i 200 filter istifadə edən, hər biri 7×7 ölçüsündə və "valid" padding ilə bir convolutional layer ilə əvəz etsək nə baş verdiyinə baxaq. Bu layer 200 feature map çıxaracaq, hər biri 1×1 (çünki kernel tam olaraq giriş feature map-lərinin ölçüsündədir və "valid" padding istifadə edirik). Başqa sözlə, o, dense layer-in etdiyi kimi 200 rəqəm çıxaracaq; və convolutional layer-in yerinə yetirdiyi hesablamalara diqqətlə baxsanız, bu rəqəmlərin dense layer-in istehsal etdiyi rəqəmlərlə tam olaraq eyni olacağını görə bilərsiniz. Yeganə fərq odur ki, dense layer-in çıxışı [batch size, 200] formalı bir tensor idi, convolutional layer isə [batch size, 1, 1, 200] formalı bir tensor çıxaracaq.

İPUCU

Bir dense layer-i convolutional layer-ə çevirmək üçün convolutional layer-dəki filter-lərin sayı dense layer-dəki vahidlərin sayına bərabər olmalıdır, filter ölçüsü giriş feature map-lərinin

³⁰Jonathan Long et al., "Fully Convolutional Networks for Semantic Segmentation", Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2015): 3431–3440.

ölçüsünə bərabər olmalıdır və "valid" padding istifadə etməlisiniz. Tezliklə görəcəyiniz kimi, stride 1 və ya daha çox təyin oluna bilər.

Bu nə üçün vacibdir? Dense layer müəyyən bir giriş ölçüsü gözlədiyi halda (çünki onun hər giriş xüsusiyyəti üçün bir çəkisi var), convolutional layer istənilən ölçüdə şəkilləri məmnuniyyətlə emal edəcək³¹ (lakin o, girişlərinin müəyyən sayda kanala malik olmasını gözləyir, çünki hər kernel hər giriş kanalı üçün fərqli bir çəki dəsti ehtiva edir). FCN yalnız convolutional layer-lərdən (və eyni xüsusiyyətə malik pooling layer-lərdən) ibarət olduğu üçün, o, istənilən ölçüdə şəkillər üzərində öyrədilə və icra oluna bilər!

Məsələn, fərz edək ki, gül təsnifatı və lokalizasiyası üçün artıq bir CNN öyrətmişik. O, 224×224 şəkillər üzərində öyrədilib və 10 rəqəm çıxarır:

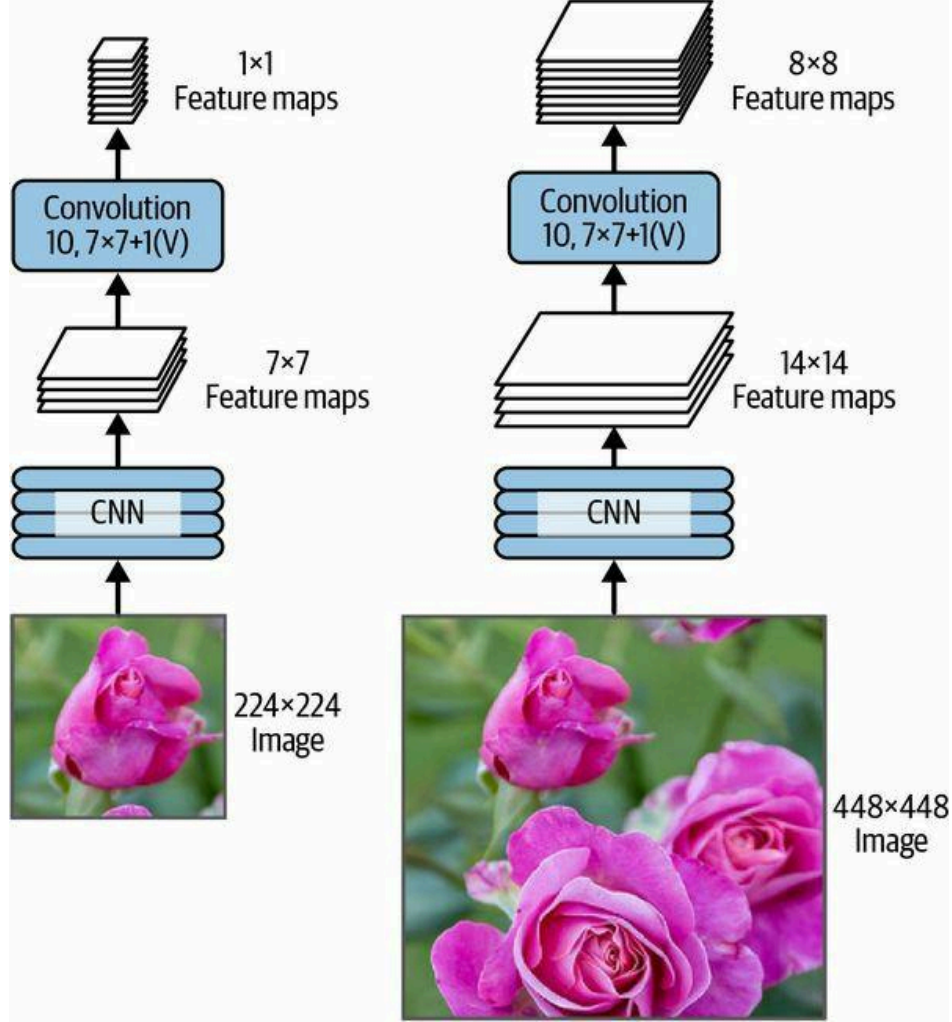
- 0-dan 4-ə qədər çıxışlar softmax activation function-dan keçirilir və bu, sinif ehtimallarını verir (sinif başına bir dənə).
- 5-ci çıxış sigmoid activation function-dan keçirilir və bu, objectness score-u verir.
- 6-cı və 7-ci çıxışlar bounding box-un mərkəz koordinatlarını təmsil edir; onlar da 0-dan 1-ə qədər dəyişmələrini təmin etmək üçün sigmoid activation function-dan keçir.
- Nəhayət, 8-ci və 9-cu çıxışlar bounding box-un hündürlüyünü və enini təmsil edir; bounding box-ların şəklin sərhədlərindən kənara çıxmasına imkan vermək üçün onlar heç bir activation function-dan keçmir.

İndi CNN-in dense layer-lərini convolutional layer-lərə çevirə bilərik. Əslində, onu yenidən öyrətməyə belə ehtiyacımız yoxdur; sadəcə çəkiliyi dense layer-lərdən convolutional layer-lərə köçürə bilərik! Alternativ olaraq, CNN-i təlimdən əvvəl FCN-ə çevirə bilərdik.

İndi fərz edək ki, çıxış layer-indən əvvəlki son convolutional layer (həm də bottleneck layer adlanır) şəbəkəyə 224×224 şəkil verildikdə 7×7 feature map-lər çıxarır (bax: Şəkil 14-26-nın sol tərəfi). FCN-ə 448×448 şəkil versək (bax: Şəkil 14-26-nın sağ tərəfi), bottleneck layer indi 14×14 feature map-lər çıxaracaq. Dense çıxış layer-i "valid" padding və stride 1 ilə 7×7 ölçülü 10 filter istifadə edən bir convolutional layer ilə əvəz olunduğu üçün, çıxış 10 feature map-dən ibarət olacaq, hər biri 8×8 ölçüsündə (çünki $14 - 7 + 1 = 8$).³² Başqa sözlə, FCN bütün şəkli yalnız bir dəfə emal edəcək və hər xanası 10 rəqəm (5 sinif ehtimalı, 1 objectness score və 4 bounding box koordinatı) ehtiva edən 8×8 şəbəkə çıxaracaq. Bu, tam olaraq orijinal CNN-i götürüb sətir başına 8 addım və sütun başına 8 addımla şəkil boyunca sürüşdürmək kimidir. Bunu görüntüləmək üçün, orijinal şəkli 14×14 şəbəkəyə bölüb, sonra bu şəbəkə boyunca 7×7 pəncərə sürüşdürdüyünüzü təsəvvür edin; pəncərə üçün $8 \times 8 = 64$ mümkün yer olacaq, deməli 8×8 proqnoz. Lakin FCN yanaşması *daha* effektivdir, çünki şəbəkə şəkli yalnız bir dəfə baxır. Əslində, *You Only Look Once* (YOLO) çox populyar bir object detection arxitekturasının adıdır ki, ona növbəti baxacağıq.

³¹Bir kiçik istisna var: "valid" padding istifadə edən convolutional layer giriş ölçüsü kernel ölçüsündən kiçik olarsa, xəta verəcək.

³²Bu, şəbəkədə yalnız "same" padding istifadə etdiyimizi fərz edir: "valid" padding feature map-lərin ölçüsünü azaldardı. Üstəlik, 448 heç bir yuvarlaqlaşdırma xətası olmadan 7-yə çatana qədər bir neçə dəfə 2-yə bölünə bilər. Hər hansı layer 1 və ya 2-dən fərqli stride istifadə edərsə, müəyyən yuvarlaqlaşdırma xətası ola bilər, beləliklə feature map-lər yenə də daha kiçik ola bilər.



Şəkil 14-26. Eyni fully convolutional network kiçik şəkli (solda) və böyük şəkli (sağda) emal edir

You Only Look Once (YOLO)

YOLO Joseph Redmon və başqaları tərəfindən 2015-ci ildə bir məqalədə təklif olunan sürətli və dəqiq bir object detection arxitekturasıdır.³³ O, o qədər sürətlidir ki, Redmon-un demosunda göründüyü kimi, bir video üzərində real vaxtda işləyə bilər. YOLO-nun arxitekturası indicə müzakirə etdiyimizə kifayət qədər bənzəyir, lakin bir neçə mühüm fərqlə:

- Hər şəbəkə xanası üçün YOLO yalnız bounding box mərkəzi həmin xananın içində yerləşən obyektləri nəzərə alır. Bounding box koordinatları həmin xanaya nisbətəndir, burada (0, 0) xananın yuxarı-sol küncü, (1, 1) isə aşağı-sağ küncü deməkdir. Lakin bounding box-un hündürlüyü və eni xanadan xeyli kənara çıxa bilər.
- O, hər şəbəkə xanası üçün (yalnız bir deyil) iki bounding box çıxarır ki, bu da modelə iki obyektin bir-birinə o qədər yaxın olduğu və onların bounding box mərkəzlərinin eyni xananın içində yerləşdiyi halları idarə etməyə imkan verir. Hər bounding box öz objectness score-u ilə də gəlir.

³³Joseph Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection", Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2016): 779–788.

- YOLO həmçinin hər şəbəkə xanası üçün bir sinif ehtimal paylanması (class probability distribution) çıxarır və şəbəkə xanası başına 20 sinif ehtimalı proqnozlaşdırır, çünki YOLO 20 sinif ehtiva edən PASCAL VOC dataseti üzərində öyrədilib. Bu, kobud bir sinif ehtimal xəritəsi (class probability map) yaradır. Diqqət edin ki, model bounding box başına deyil, şəbəkə xanası başına bir sinif ehtimal paylanması proqnozlaşdırır. Lakin son əməl zamanı hər bounding box üçün sinif ehtimallarını qiymətləndirmək mümkündür — hər bounding box-un sinif ehtimal xəritəsindəki hər sinfə nə dərəcədə uyğun gəldiyini ölçməklə. Məsələn, bir avtomobilin qarşısında dayanmış bir insanın şəklini təsəvvür edin. İki bounding box olacaq: biri avtomobil üçün böyük üfüqi olan, biri də insan üçün daha kiçik şaquli olan. Bu bounding box-ların mərkəzləri eyni şəbəkə xanasının içində ola bilər. Bəs hər bounding box-a hansı sinfin təyin olunmalı olduğunu necə biləcəyik? Sinif ehtimal xəritəsi “car” sinfinin üstünlük təşkil etdiyi böyük bir bölgə ehtiva edəcək və onun içində “person” sinfinin üstünlük təşkil etdiyi daha kiçik bir bölgə olacaq. Ümid olunur ki, avtomobilin bounding box-u təxminən “car” bölgəsinə, insanın bounding box-u isə təxminən “person” bölgəsinə uyğun gələcək: bu, hər bounding box-a düzgün sinfin təyin olunmasına imkan verəcək.

YOLO ilkin olaraq Joseph Redmon tərəfindən C dilində hazırlanmış açıq mənbəli bir dərin öyrənmə çərçivəsi olan Darknet istifadə edərək hazırlanıb, lakin tezliklə TensorFlow, Keras, PyTorch və daha çoxuna köçürülüb. O, illər ərzində davamlı olaraq təkmilləşdirilib — YOLOv2, YOLOv3 və YOLO9000 (yenə Joseph Redmon və başqaları tərəfindən), YOLOv4 (Alexey Bochkovskiy və başqaları tərəfindən), YOLOv5 (Glenn Jocher tərəfindən) və PP-YOLO (Xiang Long və başqaları tərəfindən).

Hər versiya müxtəlif texnikalardan istifadə edərək sürət və dəqiqlikdə təsirli təkmilləşdirmələr gətirdi; məsələn, YOLOv3 dəqiqliyi qismən *anchor priors* sayəsində artırdı — bəzi bounding box formalarının sinifdən asılı olaraq digərlərindən daha ehtimallı olması faktından istifadə edərək (məsələn, insanların şaquli bounding box-ları olur, avtomobillərin isə adətən yox). Onlar həmçinin şəbəkə xanası başına bounding box-ların sayını artırdılar, daha çox sinifli müxtəlif dataset-lər üzərində öyrətdilər (YOLO9000 vəziyyətində iyerarxiyada təşkil olunmuş 9.000-ə qədər sinif), CNN-də itirilən məkan ayırdetməsinin (spatial resolution) bir hissəsini bərpa etmək üçün skip connection-lar əlavə etdilər (bunu tezliklə, semantic segmentation-a baxanda müzakirə edəcəyik) və daha çoxunu. Bu modellərin də bir çox variantı var, məsələn YOLOv4-tiny, daha az güclü maşınlar üzərində öyrədilmək üçün optimallaşdırılıb və son dərəcə sürətli işləyə bilər (saniyədə 1.000-dən çox kadr!), lakin bir qədər aşağı mean average precision (mAP) ilə.

MEAN AVERAGE PRECISION

Object detection tapşırıqlarında çox geniş yayılmış bir metrik mean average precision-dur. “Mean average” bir az təkrar səslənir, elə deyilmi? Bu metriki başa düşmək üçün Fəsil 3-də müzakirə etdiyimiz iki təsnifat metrikinə qayıdaq: precision və recall. Mübadiləni (trade-off) xatırlayın: recall nə qədər yüksəkdirsə, precision bir o qədər aşağıdır. Bunu bir precision/recall əyrisində (curve) görüntüləyə bilərsiniz (bax: Şəkil 3-6). Bu əyrini tək bir rəqəmə yığmaq üçün onun əyri altındakı sahəsinə (area under the curve, AUC) hesablaya bilərdik. Lakin diqqət edin ki, precision/recall əyrisi precision-ın recall artdıqca əslində yüksəldiyi bir neçə bölgə ehtiva edə bilər, xüsusən aşağı recall dəyərlərində (bunu Şəkil 3-6-nın yuxarı sol hissəsində görə bilərsiniz). Bu, mAP metriki üçün motivasiyalardan biridir.

Fərz edək ki, təsnifatçının 10% recall-da 90% precision-u, lakin 20% recall-da 96% precision-u var. Burada əslində heç bir mübadilə yoxdur: təsnifatçını 10% recall əvəzinə 20% recall-da istifadə etmək daha məntiqlidir, çünki həm daha yüksək recall, həm də daha yüksək precision əldə edəcəksiniz. Beləliklə, 10% recall-dakı precision-a baxmaq əvəzinə, əslində ən azı 10% recall ilə təsnifatçının təklif edə biləcəyi maksimum precision-a baxmalıyıq. Bu, 90% deyil, 96% olardı. Buna görə, modelin performansı haqqında ədalətli təsəvvür əldə etməyin bir yolu ən azı 0% recall, sonra 10% recall, 20% və s. 100% recall-a qədər əldə edə biləcəyiniz maksimum precision-u hesablamaq, sonra bu maksimum precision-ların mean-ini hesablamaqdır. Bu, average precision (AP) metriki adlanır. İndi ikidən çox sinif olduqda, hər sinif üçün AP-ni hesablaya, sonra mean AP-ni (mAP) hesablaya bilərik. Vəssalam!

Object detection sistemində əlavə bir mürəkkəblik səviyyəsi var: sistem düzgün sinfi aşkarladı, lakin səhv yerdə (yəni bounding box tamamilə yanlışdırsa) nə etməli? Əlbəttə ki, bunu müsbət proqnoz kimi saymamalıyıq. Bir yanaşma IoU həddi (threshold) təyin etməkdir: məsələn, yalnız IoU müəyyən bir dəyərdən, deyək ki, 0,5-dən böyükdürsə və proqnozlaşdırılan sinif düzgündürsə, proqnozun düzgün olduğunu hesab edə bilərik. Müvafiq mAP adətən mAP@0.5 (və ya mAP@50%, yaxud bəzən sadəcə AP₅₀) kimi qeyd olunur. Bəzi yarışmalarda (məsələn, PASCAL VOC çağırışı) bunu edirlər. Digərlərində (məsələn, COCO yarışması) mAP müxtəlif IoU həddləri (0,50, 0,55, 0,60, ..., 0,95) üçün hesablanır və yekun metrik bütün bu mAP-ların mean-idir (mAP@[.50:.95] və ya mAP@[.50:0.05:.95] kimi qeyd olunur). Bəli, bu, bir mean mean average-dir.

TensorFlow Hub-da bir çox object detection modeli mövcuddur, çox vaxt pretrained çəkilərlə, məsələn YOLOv5,³⁴ SSD,³⁵ Faster R-CNN³⁶ və EfficientDet.³⁷

SSD və EfficientDet YOLO-ya bənzər “look once” aşkarlama modelləridir. EfficientDet EfficientNet convolutional arxitekturasına əsaslanır. Faster R-CNN daha mürəkkəbdir: şəkil əvvəlcə bir CNN-dən keçir, sonra çıxış obyekt ehtiva etmə ehtimalı ən yüksək olan bounding box-ları təklif edən bir region proposal network-ə (RPN) ötürülür; sonra CNN-in kəsilmiş çıxışına əsasən hər bounding box üçün bir təsnifatçı işlədilir. Bu modellərdən istifadəyə başlamaq üçün ən yaxşı yer TensorFlow Hub-ın əla object detection dərslidir.

İndiyə qədər yalnız tək şəkillərdə obyektlərin aşkarlanmasını nəzərdən keçirdik. Bəs videolar? Obyektlər təkcə hər kadrda aşkarlanmamalı, həm də zaman ərzində izlənməlidir (tracked). Gəlin indi object tracking-ə qısaca baxaq.

Object Tracking

Object tracking çətin bir tapşırıqdır: obyektlər hərəkət edir, kameraya yaxınlaşdıqca və ya ondan uzaqlaşdıqca böyüyə və ya kiçilə bilər, fırlandıqca və ya fərqli işıqlandırma şəraitinə yaxud fonlara keçdikcə görünüşləri dəyişə bilər, başqa obyektlər tərəfindən müvəqqəti olaraq örtülə

³⁴YOLOv3, YOLOv4 və onların tiny variantlarını TensorFlow Models layihəsində tapa bilərsiniz: <https://homl.info/yolotf>.

³⁵Wei Liu et al., "SSD: Single Shot Multibox Detector", Proceedings of the 14th European Conference on Computer Vision 1 (2016): 21–37.

³⁶Shaoqing Ren et al., "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", Proceedings of the 28th International Conference on Neural Information Processing Systems 1 (2015): 91–99.

³⁷Mingxing Tan et al., "EfficientDet: Scalable and Efficient Object Detection", arXiv preprint arXiv:1911.09070 (2019).

(occluded) bilər və s. Ən populyar object tracking sistemlərindən biri DeepSORT-dur.³⁸ O, klassik alqoritmlərin və dərin öyrənmənin birləşməsinə əsaslanır:

- O, əvvəlki aşkarlamalar əsasında və obyektlərin sabit sürətlə hərəkət etdiyini fərz edərək bir obyektin ən ehtimallı cari mövqeyini qiymətləndirmək üçün Kalman filter-lərindən istifadə edir.
- O, yeni aşkarlamalarla mövcud izlənən obyektlər arasındakı oxşarlığı (resemblance) ölçmək üçün bir dərin öyrənmə modelindən istifadə edir.
- Nəhayət, o, yeni aşkarlamaları mövcud izlənən obyektlərlə (və ya yeni izlənən obyektlərlə) uyğunlaşdırmaq üçün Hungarian alqoritmindən istifadə edir: bu alqoritm aşkarlamalarla izlənən obyektlərin proqnozlaşdırılan mövqeləri arasındakı məsafəni minimuma endirən, eyni zamanda görünüş uyğunsuzluğunu da minimuma endirən uyğunlaşdırma kombinasiyasını effektiv şəkildə tapır.

Məsələn, əks istiqamətdə hərəkət edən mavi topdan indicə sıçramış qırmızı bir topu təsəvvür edin. Topların əvvəlki mövqeləri əsasında Kalman filter topların bir-birinin içindən keçəcəyini proqnozlaşdıracaq: həqiqətən, o, obyektlərin sabit sürətlə hərəkət etdiyini fərz edir, ona görə də sıçramanı gözləməyəcək. Əgər Hungarian alqoritm yalnız mövqeləri nəzərə alsaydı, o, yeni aşkarlamaları sanki toplar bir-birinin içindən keçib rəngləri dəyişibləmiş kimi səhv toplara məmənuniyyətlə uyğunlaşdırardı. Lakin oxşarlıq ölçüsü sayəsində Hungarian alqoritm problemi sezsək. Toplar bir-birinə çox bənzəmədiyini fərz etsək, alqoritm yeni aşkarlamaları düzgün toplara uyğunlaşdıracaq.

İPUCU

GitHub-da bir neçə DeepSORT tətbiqi mövcuddur, o cümlədən YOLOv4 + DeepSORT-un TensorFlow tətbiqi: <https://github.com/theAIGuysCode/yolov4-deepsort>.

İndiyə qədər obyektləri bounding box-lardan istifadə edərək lokalizasiya etdik. Bu, çox vaxt kifayətdir, lakin bəzən obyektləri daha böyük dəqiqliklə lokalizasiya etmək lazımdır — məsələn, videokonfrans zəngi zamanı bir insanın arxasındakı fonu silmək üçün. Gəlin piksel səviyyəsinə necə enəcəyimizə baxaq.

Semantic Segmentation

Semantic segmentation-da hər piksel aid olduğu obyektin sinfinə görə təsnif olunur (məsələn, yol, avtomobil, piyada, bina və s.), Şəkil 14-27-də göstərilirdi kimi. Diqqət edin ki, eyni sinfin fərqli obyektləri *fərqləndirilmir*. Məsələn, seqmentləşdirilmiş şəklin sağ tərəfindəki bütün velosipedlər bir böyük piksel topası kimi qalır. Bu tapşırıqdakı əsas çətinlik odur ki, şəkillər adi bir CNN-dən keçdikdə onlar tədricən məkan ayırdetməsini (spatial resolution) itirir (stride-ı 1-dən böyük olan layer-lər səbəbindən); ona görə də adi bir CNN şəklin aşağı solunda harada isə bir insanın olduğunu bilə bilər, lakin bundan daha dəqiq ola bilməyəcək.

³⁸Nicolai Wojke et al., "Simple Online and Realtime Tracking with a Deep Association Metric", arXiv preprint arXiv:1703.07402 (2017).



Şəkil 14-27. Semantic segmentation

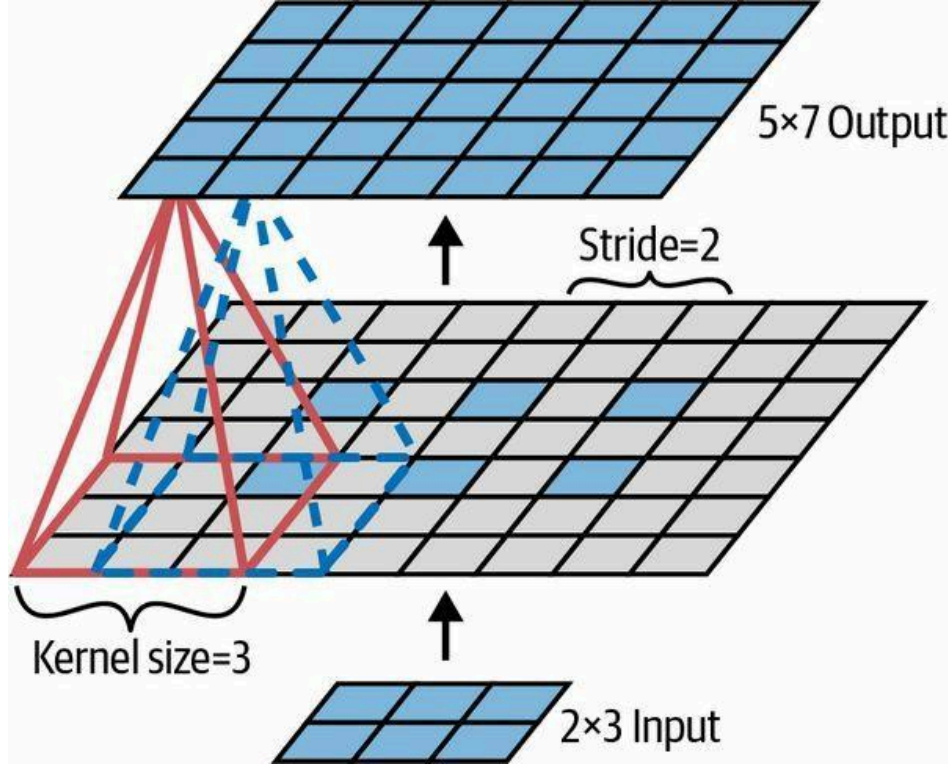
Object detection-da olduğu kimi, bu problemi həll etmək üçün bir çox fərqli yanaşma var, bəziləri kifayət qədər mürəkkəbdir. Lakin əvvəl qeyd etdiyim Jonathan Long və başqalarının 2015-ci il məqaləsində fully convolutional network-lər haqqında kifayət qədər sadə bir həll təklif olundu. Müəlliflər pretrained bir CNN götürüb onu FCN-ə çevirməklə başlayır. CNN giriş şəklinə ümumi 32 stride tətbiq edir (yəni 1-dən böyük bütün stride-ları toplasanız), yəni son layer giriş şəklindən 32 dəfə kiçik feature map-lər çıxarır. Bu, açıq-aşkar çox kobuddur, ona görə də onlar ayırdetməni 32 dəfə artıran tək bir *upsampling layer* əlavə etdilər.

Upsampling üçün (bir şəklın ölçüsünü artırmaq) bir neçə həll mövcuddur, məsələn bilinear interpolation, lakin bu, yalnız $\times 4$ və ya $\times 8$ -ə qədər kifayət qədər yaxşı işləyir. Bunun əvəzinə onlar *transposed convolutional layer*-dən istifadə edir:³⁹ bu, əvvəlcə şəklı boş sətirlər və sütunlar (sıfırlarla dolu) əlavə etməklə dartmağa (stretching), sonra adi bir convolution yerinə yetirməyə ekvivalentdir (bax: Şəkil 14-28). Alternativ olaraq, bəziləri bunu kəsr stride-lar (fractional strides) istifadə edən adi bir convolutional layer kimi düşünməyə üstünlük verir (məsələn, Şəkil 14-28-də stride $1/2$ -dir). Transposed convolutional layer xətti interpolasiyaya yaxın bir şey yerinə yetirmək üçün ilkinləşdirilə bilər, lakin o, öyrənilə bilən bir layer olduğu üçün, təlim zamanı daha yaxşısını etməyi öyrənəcək. Keras-da Conv2DTranspose layer-indən istifadə edə bilərsiniz.

QEYD

Transposed convolutional layer-də stride girişin nə qədər dartılacağını müəyyən edir, filter addımlarının ölçüsünü deyil, ona görə də stride nə qədər böyükdürsə, çıxış bir o qədər böyükdür (convolutional layer-lərdən və ya pooling layer-lərdən fərqli olaraq).

³⁹Bu tip layer bəzən deconvolution layer adlandırılır, lakin riyaziyyatçıların deconvolution adlandırdığı əməliyyatı yerinə yetirmir, ona görə də bu addan çəkinmək lazımdır.



Şəkil 14-28. Transposed convolutional layer ilə upsampling

DİĞƏR KERAS CONVOLUTIONAL LAYER NÖVLƏRİ

Keras həmçinin bir neçə başqa convolutional layer növü təklif edir:

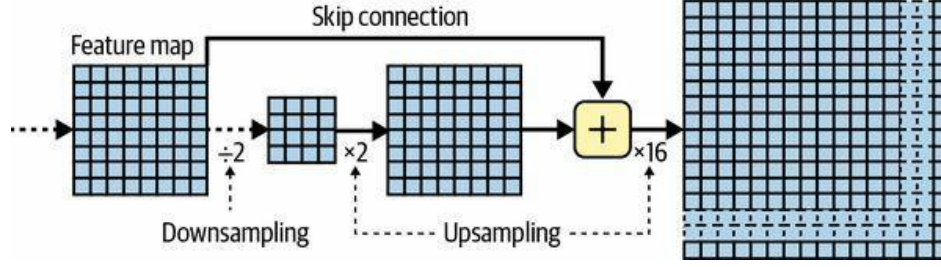
`tf.keras.layers.Conv1D` — Zaman sıraları (time series) və ya mətn (hərflərin və ya sözlərin ardıcılığı) kimi 1D girişlər üçün convolutional layer, Fəsil 15-də görəcəyiniz kimi.

`tf.keras.layers.Conv3D` — 3D PET skanları kimi 3D girişlər üçün convolutional layer.

`dilation_rate` — İstənilən convolutional layer-in `dilation_rate` hyperparametrini 2 və ya daha çox dəyərə təyin etmək *à-trous convolutional layer* yaradır (“à trous” fransızca “deşiklərlə” deməkdir). Bu, sətirlər və sütunlar (yənideşiklər) əlavə etməklə genişləndirilmiş (dilated) bir filter istifadə edən adi bir convolutional layer-ə ekvivalentdir. Məsələn, `[[1,2,3]]`-ə bərabər 1×3 filter dilation rate 4 ilə genişləndirilərək `[[1, 0, 0, 0, 2, 0, 0, 0, 3]]` genişləndirilmiş filteri ilə nəticələne bilər. Bu, convolutional layer-ə heç bir hesablama qiyməti olmadan və əlavə parametr istifadə etmədən daha böyük bir receptive field-ə malik olmağa imkan verir.

Upsampling üçün transposed convolutional layer-lərdən istifadə yaxşıdır, lakin yenə də çox qeyri-dəqiqdir. Daha yaxşısını etmək üçün Long və başqaları aşağı layer-lərdən skip connection-lar əlavə etdilər: məsələn, onlar çıxış şəklini 32 əvəzinə 2 dəfə upsample etdilər və bu ikiqat ayırdetməyə malik aşağı bir layer-in çıxışını əlavə etdilər. Sonra nəticəni 16 dəfə upsample etdilər ki, bu da ümumi 32 dəfə upsampling faktoruna gətirib çıxardı (bax: Şəkil 14-29). Bu, əvvəlki pooling layer-lərdə itirilən məkan ayırdetməsinin bir hissəsini bərpa etdi. Ən yaxşı arxitekturalarında daha da aşağı bir layer-dən daha incə detalları bərpa etmək üçün ikinci oxşar bir skip connection istifadə etdilər. Qısaca, orijinal CNN-in çıxışı aşağıdakı əlavə addımlardan keçir: $\times 2$ upsample et, aşağı bir layer-in çıxışını (uyğun miqyasda) əlavə et, $\times 2$ upsample et, daha da aşağı bir layer-in çıxışını əlavə et və nəhayət $\times 8$ upsample et. Hətta

orijinal şəklın ölçüsündən kənara miqyaslandırmaq da mümkündür: bu, bir şəklın ayırdetməsini artırmaq üçün istifadə oluna bilər ki, bu da *super-resolution* adlanan bir texnikadır.



Şəkil 14-29. Skip layer-lər aşağı layer-lərdən bir qədər məkan ayırdetməsini (spatial resolution) bərpa edir

Instance segmentation semantic segmentation-a bənzəyir, lakin eyni sinfin bütün obyektlərini bir böyük topaya birləşdirmək əvəzinə, hər obyekt digərlərindən fərqləndirilir (məsələn, o, hər fərdi velosipedi müəyyən edir). Məsələn, 2017-ci ildə Kaiming He və başqalarının bir məqaləsində təklif olunan Mask R-CNN arxitekturası Faster R-CNN modelini hər bounding box üçün əlavə olaraq bir piksel maskası (pixel mask) istehsal edərək genişləndirir.⁴⁰ Beləliklə, hər obyektin ətrafında bir dəstə təxmin olunan sinif ehtimalı ilə bir bounding box əldə etməklə yanaşı, bounding box-da obyektə aid olan piksellərin yerini müəyyən edən bir piksel maskası da əldə edirsiniz. Bu model TensorFlow Hub-da COCO 2017 dataseti üzərində pretrained mövcuddur. Lakin sahə sürətlə inkişaf edir, ona görə də ən son və ən yaxşı modelləri sınaq etmək istəyirsinizsə, <https://paperswithcode.com> saytının state-of-the-art bölməsinə baxın.

Gördüyünüz kimi, dərin computer vision sahəsi geniş və sürətli templidir, hər il hər cür arxitektura ortaya çıxır. Demək olar ki, hamısı convolutional neural network-lərə əsaslanır, lakin 2020-ci ildən bəri computer vision sahəsinə başqa bir neural net arxitekturası daxil olub: transformer-lər (bunları Fəsil 16-da müzakirə edəcəyik). Son onillikdə əldə olunan tərəqqi heyranəmiz olub və tədqiqatçılar indi getdikcə daha çətin problemlərə diqqət yetirirlər, məsələn adversarial learning (şəbəkəni onu aldatmaq üçün hazırlanmış şəkillərə qarşı daha davamlı etməyə çalışır), explainability (şəbəkənin niyə müəyyən bir təsnifat etdiyini anlamaq), realistik şəkil generasiyası (buna Fəsil 17-də qayıdacağıq), single-shot learning (bir obyekti yalnız bir dəfə gördükdən sonra tanıya bilən sistem), videodakı növbəti kadrları proqnozlaşdırmaq, mətn və şəkil tapşırıqlarını birləşdirmək və daha çoxu.

İndi növbəti fəslə keçirik, orada təkrar olunan neural network-lər (recurrent neural networks) və convolutional neural network-lər istifadə edərək zaman sıraları (time series) kimi ardıcıl məlumatları necə emal edəcəyimizə baxacağıq.

Çalışmalar

1. CNN-in şəkil təsnifatı üçün fully connected DNN üzərində üstünlükləri hansılardır?
2. Hər biri 3×3 kernel-li, stride 2 və "same" padding-li üç convolutional layer-dən ibarət bir CNN-i nəzərdən keçirin. Ən aşağı layer 100 feature map, ortadakı 200, üstdəki isə 400 feature map çıxarır. Giriş şəkilləri 200×300 piksellə RGB şəkillərdir:

⁴⁰Kaiming He et al., "Mask R-CNN", arXiv preprint arXiv:1703.06870 (2017).

- a. CNN-də parametrlərin ümumi sayı neçədir?
 - b. 32 bitlik float-lar istifadə edirsinizsə, bu şəbəkə tək bir instans üçün proqnoz verərkən ən azı nə qədər RAM tələb edəcək?
 - c. Bəs 50 şəkildən ibarət bir mini-batch üzərində öyrədərkən?
3. Bir CNN-i öyrədərkən GPU-nuzun yaddaşı tükənərsə, problemi həll etmək üçün cəhd edə biləcəyiniz beş şey hansılardır?
4. Niyə eyni stride-lı convolutional layer əvəzinə max pooling layer əlavə etmək istəyərdiniz?
5. Local response normalization layer-i nə vaxt əlavə etmək istəyərdiniz?
6. AlexNet-in LeNet-5 ilə müqayisədə əsas yeniliklərini adlandırın bilirsinizmi? Bəs GoogLeNet, ResNet, SNet, Xception və EfficientNet-in əsas yenilikləri?
7. Fully convolutional network nədir? Bir dense layer-i convolutional layer-ə necə çevirə bilərsiniz?
8. Semantic segmentation-in əsas texniki çətinliyi nədir?
9. Öz CNN-inizi sıfırdan qurun və MNIST üzərində mümkün olan ən yüksək dəqiqliyə nail olmağa çalışın.
10. Böyük şəkil təsnifatı üçün transfer learning istifadə edin, bu addımlardan keçərək:
 - a. Sınıf başına ən azı 100 şəkil ehtiva edən bir training set yaradın. Məsələn, öz şəkillərinizi yerə (çimərlik, dağ, şəhər və s.) görə təsnif edə bilərsiniz, ya da mövcud bir datasetdən (məsələn, TensorFlow Datasets-dən) istifadə edə bilərsiniz.
 - b. Onu training set, validation set və test set-ə bölün.
 - c. Giriş pipeline-ını qurun, müvafiq ön emal əməliyyatlarını tətbiq edin və istəyə bağlı olaraq data augmentation əlavə edin.
 - d. Bu dataset üzərində pretrained bir modeli fine-tune edin.
11. TensorFlow-un Style Transfer dərslərindən keçin. Bu, dərin öyrənmədən istifadə edərək sənət yaratmağın əyləncəli bir yoludur.

Bu çalışmaların həlləri bu fəslin notebook-unun sonunda, <https://homl.info/colab3> ünvanında mövcuddur.